

Apuntes de Estructuras de Datos y Algoritmos (EDA)

Grado en Ingeniería en Inteligencia Artificial

Agustín Daniel Delgado Muñoz

Fernando López Ostenero

Octubre de 2026

Índice

Índice de tablas	III
Índice de figuras	V
1. Recursión	1
1.1. Noción de recursividad	1
1.1.1. Estructura general de un algoritmo recursivo	2
1.1.2. Diferentes tipos de algoritmos recursivos	3
1.1.3. Problemas de la recursión	8
1.1.4. Diseño por contrato	10
1.2. Ejemplos de Algoritmos Numéricos	13
1.3. Ejemplos de Estructuras de Datos	16
1.3.1. La cola para pagar en el supermercado	16
1.3.2. Las carpetas del ordenador	17
1.3.3. Definiciones recursivas de estructuras de datos	18
2. Introducción al Coste Computacional	19
2.1. Modelo RAM	19
2.1.1. Características del Modelo RAM	20
2.1.2. Operaciones elementales del Modelo RAM	20
2.1.3. Cálculo del coste temporal en el Modelo RAM	21
2.2. Función de coste temporal de un algoritmo	23
2.3. Notación de Landau	27
2.3.1. Definición de la notación de Landau	28
2.3.2. Álgebra de la notación de Landau	30
2.3.3. Jerarquía de órdenes de complejidad	32

2.4. Cálculo del coste computacional de algoritmos 34

2.4.1. Coste computacional de algoritmos iterativos 35

2.4.2. Coste computacional de algoritmos recursivos 39

Bibliografía **45**

Índice de tablas

- 2.1. Contribución de cada término en $T_{\mathcal{A}}(n) = n^2 + 20 \cdot n + 1500$ (valores y porcentajes). 28

Índice de figuras

1.1. Un problema expresado recursivamente	1
1.2. No podemos seguir aplicando recursión	1
1.3. Recursión lineal: secuencia de llamadas recursivas.	4
1.4. Recursión múltiple: árbol de llamadas recursivas.	5
1.5. Llamadas recursivas y valores devueltos por la función factorial.	7
1.6. Llamadas recursivas de factorialF.	7

Capítulo 1

Recursión

Cuando hablamos de recursividad, seguro que se nos viene a la cabeza ese chiste tan conocido que dice que “*para entender la recursividad, primero hay que entender la recursividad*”. Aunque no le falta razón, vamos a ver que el chiste esconde una trampa y que si lo seguimos al pie de la letra no vamos por buen camino.

1.1. Noción de recursividad

¿Qué es la recursividad? Básicamente se trata de una forma de resolver un problema que puede ser expresado en base a **una versión más sencilla de sí mismo**.

Pongamos un ejemplo: nuestro problema va a consistir en sumar todos los elementos de una secuencia de números. Podemos expresarlo de forma recursiva como sumar el primer número más el resultado de sumar todos los elementos de la secuencia de números resultantes de eliminar el primero, tal y como se muestra en la figura 1.1.

Sumar

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

Se puede expresar como:

1 + Sumar

2	3	4	5	6	7	8
---	---	---	---	---	---	---

Figura 1.1: Un problema expresado recursivamente

Pero aquí nos falta algo más, ya que llegaría el momento en el que llegaríamos a la situación mostrada en la figura 1.2, en la que la secuencia que nos queda por sumar es vacía. En este momento ya no podemos expresar el problema en función de una versión más reducida del mismo, ya que la secuencia vacía no tiene un primer elemento que sacar de la misma.

1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + Sumar

--

Figura 1.2: No podemos seguir aplicando recursión

Es por eso por lo que necesitamos un **caso base** que nos diga qué hacer en aquellas situaciones en las que no podemos aplicar la recursión o, incluso, cuando resolver el problema es tan sencillo que, aunque podamos definirlo en función de una versión más pequeña de sí mismo, no es necesario.

De esta forma, podríamos pensar en un caso base en el que estemos sumando una secuencia vacía. Pero, ¿cuánto suman todos los elementos de una secuencia vacía? Si no hay elementos

que sumar, la única respuesta posible es que suman 0 (que es, por cierto, el elemento neutro de la suma). Pero no hay una única respuesta correcta, ya que también podríamos pensar en que sumar el único elemento de una secuencia unitaria (que solo tiene ese elemento) es, precisamente ese elemento.

Si establecemos las secuencias con un único elemento como nuestro caso base, nos estaremos ahorrando un paso de recursión, al no tener que llegar a vaciar la secuencia. ¡Esto es genial! Sin embargo, nuestra solución al problema no podría sumar secuencias vacías y puede que en algún momento eso condujese a un error no deseado.

¿Qué podemos hacer? Pues nada nos impide que en un algoritmo recursivo haya más de un caso base. Así, podríamos expresar nuestra solución tal y como se refleja en el algoritmo 1.

Algoritmo 1 Pseudocódigo con el algoritmo recursivo para sumar los elementos de una secuencia de números.

```

1: Función SUMARSECUENCIA( $S$ )
2:   Si esVacia( $S$ ) entonces
3:     devolver 0
4:   Si no Si esUnitaria( $S$ ) entonces
5:     devolver primerElemento( $S$ )
6:   Si no
7:     devolver primerElemento( $S$ ) + sumarSecuencia(resto( $S$ ))
8:   Fin Si
9: Fin Función

```

Si los casos base nos permiten resolver el problema directamente, aquellos casos en los que necesitamos resolver una versión más simple del problema mediante recursión se denominan **casos recursivos**. Al igual que con los casos base, nada impide que un algoritmo recursivo tenga múltiples casos recursivos.

1.1.1. Estructura general de un algoritmo recursivo

Gracias al ejemplo anterior, ahora podemos establecer cuál es la estructura general de un algoritmo recursivo:

1. **Definición de la solución al problema:** para poder resolver nuestro problema tendremos que crear una estructura de código que pueda ser ejecutada sobre unos datos de entrada. Esto se denomina genéricamente **subprograma** y en cada lenguaje puede tener diferentes nombres (función, procedimiento, método, etc.).

En nuestro caso, en la línea 1 estamos definiendo que nuestro problema se va a resolver mediante una función (ya que devuelve un valor) a la que vamos a llamar **sumarSecuencia**. Esta función recibe un parámetro S , el cual contiene la secuencia a ser sumada.

2. **Caso(s) base:** son aquellos casos en los que podemos resolver el problema sin reducirlo a una versión más simplificada de sí mismo. Esto es, sin aplicar recursión.

En nuestro ejemplo serían las líneas 3 y 5, ya que devuelven un resultado en el que no se necesita resolver el mismo problema. Fijémonos que en estos casos base no existe una llamada recursiva a la función `sumarSecuencia`¹.

3. **Caso(s) recursivo(s)**: son aquellos casos en los que para resolver el problema necesitamos reducirlo a una versión más simplificada de sí mismo.

En nuestro ejemplo, el caso recursivo lo tenemos en la línea 7, donde vemos que se realiza una llamada a la propia función `sumarSecuencia` con una secuencia más pequeña que la de entrada.

4. **Análisis por casos**: es una estructura condicional que nos permite discernir si los datos de entrada del problema pueden ser resueltos directamente (lo que nos conduciría a un caso base) o necesitamos emplear la recursión (lo que nos conduciría a un caso recursivo).

En nuestro ejemplo, la estructura condicional *Si ... entonces ... Si no...* es la que nos permite realizar el análisis por casos. En la línea 2 comprobamos si la secuencia de entrada *S* es vacía. Si lo es, entonces estamos en un caso base y devolvemos directamente 0.

En la línea 4 comprobamos si la secuencia *S* contiene un único elemento, devolviendo su primer (y único) elemento en caso afirmativo mediante el segundo caso base.

Por último, en la línea 6 el *Si no* establece que en caso de no haberse cumplido ninguna de las condiciones anteriores, se está ante una secuencia con al menos dos elementos y el problema debe ser resuelto recursivamente. Esto nos conduce, directamente, al caso recursivo de nuestra solución.

1.1.2. Diferentes tipos de algoritmos recursivos

Aunque todo algoritmo recursivo siempre va a seguir la estructura presentada en el apartado anterior, es importante saber diferenciar unos algoritmos recursivos de otros.

Nos centraremos en dos criterios diferentes para clasificar algoritmos recursivos: según el número de llamadas recursivas y según la forma de los casos recursivos.

Según el número de llamadas recursivas

Este criterio de clasificación se basa en el número de llamadas recursivas que se realiza por cada llamada ejecutada. Por ejemplo, el algoritmo que suma los números de una secuencia (algoritmo 1) genera una única llamada recursiva por cada llamada ejecutada. Concretamente, en la línea 7 de dicho algoritmo vemos la única llamada recursiva que se realiza.

Este tipo de recursión se denomina **recursión lineal**: la cadena de llamadas recursivas puede asimilarse a una secuencia de llamadas en la que cada una tiene (como mucho) una llamada posterior. En la figura 1.3 podemos ver la cadena de llamadas que generaría la llamada inicial `sumarSecuencia([1,2,3])`.

¹Pero puede haber, como sucede en el ejemplo, llamadas a otras operaciones. Lo importante aquí es que no se llama a la propia función que estamos definiendo.

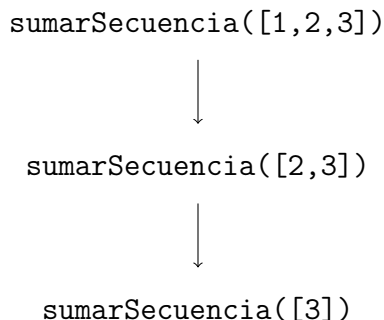


Figura 1.3: Recursión lineal: secuencia de llamadas recursivas.

Veamos otro ejemplo con el algoritmo 2 que imprime la Sucesión de Collatz² para un número n .

Algoritmo 2 Pseudocódigo con un programa recursivo que imprime la Sucesión de Collatz para un número n de partida.

```

1: Procedimiento COLLATZ( $n$ )
2:   print  $n$ 
3:   Si  $n == 1$  entonces
4:     print "Hemos llegado a 1."
5:   Si no Si  $n$  es par entonces
6:     collatz( $n/2$ )
7:   Si no
8:     collatz( $3*n+1$ )
9:   Fin Si
10: Fin Procedimiento
  
```

Si ejecutamos este algoritmo pasándole, por ejemplo, el número 10, nos imprimirá la siguiente secuencia de números:

10 5 16 8 4 2 1

y se detendrá indicando que, en efecto, hemos llegado al número 1.

Al mirar el código, vemos que en las líneas 6 y 8 hay sendas llamadas recursivas. Esto podría inducirnos a pensar que estamos ante un algoritmo recursivo que no puede clasificarse como recursión lineal, ya que hay más de una llamada recursiva.

Sin embargo, a pesar de que haya dos llamadas recursivas solo se va a ejecutar una. Ese es el detalle importante: cada ejecución del algoritmo recursivo solo provoca una única llamada recursiva. En efecto, hay dos, pero cada una de ellas está en un caso recursivo diferente,

Así pues, este algoritmo, al igual que el anterior, implementa una recursión lineal.

²La Sucesión de Collatz se define para un número n entero positivo de la siguiente forma: si n es par, el siguiente número será $n/2$, mientras que si n es impar, el siguiente número será $3n + 1$. La hipótesis, aún no probada, es que empecemos en el número que empecemos siempre acabaremos en el 1.

Vamos a ver otro algoritmo que calcula el n -ésimo número de la Sucesión de Fibonacci³ comenzando a numerar por 0.

Algoritmo 3 Pseudocódigo con un programa recursivo que calcula el n -ésimo número de la Sucesión de Fibonacci.

```

1: Función FIB( $n$ )
2:   Si  $n < 2$  entonces
3:     devolver  $n$ 
4:   Si no
5:     devolver  $\text{fib}(n-1) + \text{fib}(n-2)$ 
6:   Fin Si
7: Fin Función

```

Aquí volvemos a tener dos llamadas recursivas (ambas en la línea 5), pero en esta ocasión las dos están en el mismo (y único) caso recursivo. Este algoritmo no implementa una recursión lineal, sino una **recursión múltiple**. En esta ocasión una representación gráfica de las llamadas recursivas no se puede realizar con una secuencia, sino con una estructura denominada árbol⁴ tal y como se puede ver en la figura 1.4.

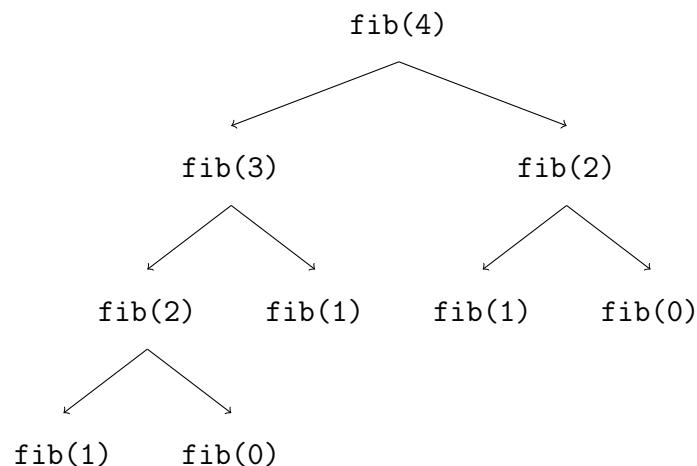


Figura 1.4: Recursión múltiple: árbol de llamadas recursivas.

Como se puede apreciar, el número de llamadas recursivas de una recursión múltiple crece mucho. Esto será de vital importancia a la hora de diseñar algoritmos recursivos y lo trataremos con más profundidad en el siguiente capítulo.

Según la forma de los casos recursivos

Vamos a comparar dos algoritmos recursivos que calculan el factorial de un número. El primero de ellos (algoritmo 4) implementa directamente la definición de factorial: el factorial de 0 es 1 y el factorial de un número n mayor que 0 es igual a n por el factorial de $n - 1$.

³La Sucesión de Fibonacci se define de la siguiente forma: el primer y segundo elementos son 0 y 1 y a partir de ahí, cada elemento se calcula como la suma de los dos anteriores. Así, la sucesión seguiría como 0 1 1 2 3 5 8 13 21 ...

⁴Como veremos más adelante, un árbol es una estructura en la que cada elemento tiene, a lo sumo, un padre y una cantidad no limitada de hijos.

Algoritmo 4 Pseudocódigo con el algoritmo recursivo no final para calcular el factorial de un número.

```

1: Función FACTORIAL(entero  $n$ )
2:   Si  $n == 0$  entonces
3:     devolver 1
4:   Si no
5:     devolver  $n * \text{factorial}(n - 1)$ 
6:   Fin Si
7: Fin Función

```

El segundo algoritmo (ver algoritmo 5) añade un segundo parámetro a modo de acumulador. Si inicializamos este parámetro a 1, entonces se cumple que:

$$\text{factorial}(n) = \text{factorialF}(n, 1)$$

Algoritmo 5 Pseudocódigo con el algoritmo recursivo final para calcular el factorial de un número.

```

1: Función FACTORIALF(entero  $n$ , entero  $f$ )
2:   Si  $n == 0$  entonces
3:     devolver  $f$ 
4:   Si no
5:     devolver  $\text{factorialF}(n - 1, n * f)$ 
6:   Fin Si
7: Fin Función

```

La diferencia entre ambos se encuentra, principalmente, en la línea 5. El primer algoritmo primero realiza la llamada recursiva para calcular el valor del factorial de $n - 1$ y después multiplica ese resultado por n para calcular el valor del factorial de n .

Para calcular, por ejemplo, $\text{factorial}(3)$, primero se va simplificando el problema como se muestra en la figura 1.5a y luego, a la vuelta de la recursión, se va construyendo la solución, tal y como se muestra en la figura 1.5b.

Sin embargo, la segunda versión primero realiza la multiplicación por n (dentro del parámetro) y luego realiza la llamada recursiva. Lo importante es que tras la llamada recursiva no se necesita realizar ningún cálculo adicional, tal y como se puede ver en la figura 1.6.

Es decir, en el primer caso al finalizar las llamadas recursivas aún no tenemos el resultado final, ya que debe ser construido a la vuelta de las llamadas. Sin embargo, en el segundo caso al finalizar las llamadas recursivas ya tenemos el valor final. En este caso, al volver de las llamadas recursivas solo se va pasando este valor, sin hacer ningún cálculo posterior.

El algoritmo `factorial` es un ejemplo de **recursión no final**, mientras que el algoritmo `factorialF` es un ejemplo de **recursión final** o **recursión de cola**. Los algoritmos con recursión final pueden optimizarse fácilmente y eliminar la recursión convirtiendo la cadena de llamadas recursivas en un bucle. Por el contrario, los algoritmos con recursión no final son más complicados de optimizar eliminando la recursión directamente.

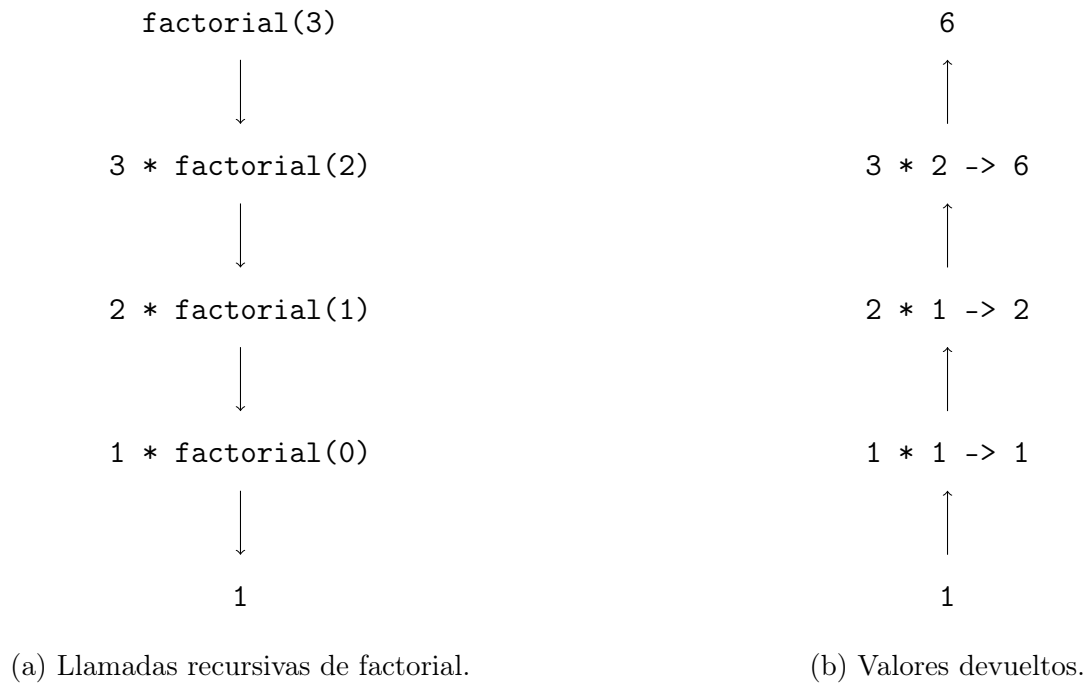


Figura 1.5: Llamadas recursivas y valores devueltos por la función factorial.

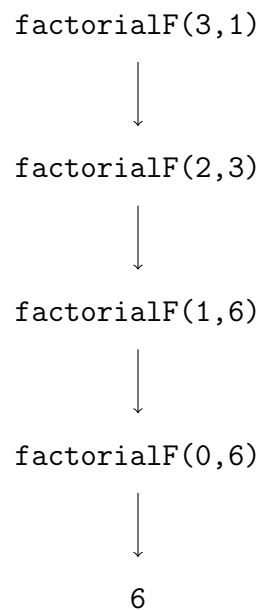


Figura 1.6: Llamadas recursivas de factorialF.

Aunque existen técnicas que permiten transformar un algoritmo con recursión no final en uno equivalente con recursión final bajo ciertas condiciones, no van a ser tratadas en esta asignatura.

1.1.3. Problemas de la recursión

Como técnica de programación, la recursión se acerca bastante a la forma de trabajar en matemáticas: encontrar la solución a nuestro problema se basa en encontrar una solución a un problema similar, pero de menor tamaño. Esto, que puede parecer ideal, acarrea una serie de problemas que debemos tener en cuenta.

Consumo de recursos

La recursión implica la ejecución de un mismo subprograma múltiples veces. Para que esto funcione correctamente, es necesario que el entorno de ejecución de nuestro programa almacene correctamente una serie de datos para cada una de estas ejecuciones.

En primer lugar, es necesario guardar el valor de los parámetros de entrada de cada una de esas ejecuciones, para que al volver de una ejecución a la que la llamó el valor de esos parámetros sea el que debe ser y podamos utilizarlo en caso de ser necesario.

Pero no solo hay que preservar esos valores para todas las llamadas, sino también el valor de todas las variables locales que se hayan definido.

Todo esto se organiza en los llamados **Registros de Activación**, que son una estructura creada en tiempo de ejecución en la que se almacena toda esa información para cada subprograma que se ejecuta.

Así, cuando un subprograma A es llamado, el entorno de ejecución reserva una zona de memoria donde se guardarán todos estos valores. Al finalizar esa ejecución de A, la memoria se liberará y se devolverá el control al subprograma que llamó al subprograma A.

Esto se organiza (de manera totalmente transparente al programador) en forma de pila⁵, en la que los registros de activación van apilándose al comenzar la ejecución de un subprograma y desapilándose cuando dicha ejecución finaliza. Esta estructura se conoce como **Pila de Llamadas**.

Así pues, al emplear la recursión, el consumo de recursos de nuestro programa sea mayor que si no la empleásemos:

- En primer lugar, cada llamada recursiva va a consumir la memoria necesaria para almacenar el registro de activación correspondiente. La pila de llamadas tiene un tamaño máximo y cuando se llena, por un excesivo número de llamadas a subprogramas, se produce un error muy típico llamado “Stack Overflow”.
- Además, hay que tener en cuenta que, probablemente⁶, en cada llamada recursiva será necesario copiar el contenido de los parámetros de entrada al registro de activación de la nueva ejecución del subprograma. Esto hace que el tiempo de ejecución de nuestro programa sea mayor por tener que realizar todas estas copias.

⁵Una pila, como veremos más adelante, es una estructura secuencial en la que los elementos entran y salen por el mismo punto. Esto provoca que el último en entrar sea el primero que puede salir, lo que da nombre a la política que sigue esta estructura: LIFO (Last In First Out).

⁶Dependerá de la forma en la que se pasen los parámetros, aunque esto queda fuera del objetivo de esta asignatura.

Es necesario tener en cuenta este consumo de recursos de cara a realizar nuestros programas recursivos. Aún así, algunos compiladores son capaces de detectar la recursión y convertirla directamente en iteración de forma que se evita este gasto de memoria y tiempo de ejecución.

Recursión infinita

Uno de los grandes problemas que nos podemos encontrar al programar recursivamente es que se genere una cadena de llamadas recursivas que jamás conducen al caso base. Esto se conoce con el nombre de **recursión infinita**.

Para ilustrarlo, recordemos el algoritmo 4 donde teníamos el pseudocódigo de una función que calcula el factorial de un número entero n ,

Aparentemente el algoritmo está bien diseñado, ya que calcula de forma correcta el factorial de cualquier número. El problema es que el factorial no está definido (como tal) para números negativos y la función factorial recibe un valor entero.

Si ejecutamos una llamada a factorial con un número no negativo, vamos a obtener correctamente el factorial de dicho número, pero si llamamos con un valor negativo, por ejemplo el -1 , eso va a provocar una cadena de llamadas recursivas que no se va a detener⁷:

$$\begin{aligned} \text{factorial}(-1) &= -1 * \text{factorial}(-2) \\ \text{factorial}(-2) &= -2 * \text{factorial}(-3) \\ \text{factorial}(-3) &= -3 * \text{factorial}(-4) \\ &\dots \end{aligned}$$

¿Qué solución tenemos ante este problema? Posiblemente en este momento hayas pensado que podemos modificar el código para devolver algo cuando el valor sea negativo. Por ejemplo, como se muestra en el algoritmo 6.

Algoritmo 6 Pseudocódigo con el algoritmo recursivo para calcular el factorial de un número, pero devolviendo 0 cuando el número es negativo.

```

1: Función FACTORIAL(entero  $n$ )
2:   Si  $n < 0$  entonces
3:     devolver 0
4:   Si no Si  $n == 0$  entonces
5:     devolver 1
6:   Si no
7:     devolver  $n * \text{factorial}(n - 1)$ 
8:   Fin Si
9: Fin Función

```

Es una posible solución, en efecto. Esto nos soluciona el error de desbordamiento de la pila de llamadas y nuestra función ya nunca va a generar una cadena (potencialmente) infinita de llamadas recursivas.

⁷Estrictamente hablando, la cadena de llamadas recursivas se detendrá en algún momento ya que, como hemos visto en la sección anterior, cada una va a generar un registro de activación en la pila de llamadas. Lo que va a suceder es que esta pila se llenará, provocando, en consecuencia, un error “Stack Overflow”.

Sin embargo, esto puede acarrear otros problemas, ya que alguien que utilice la función puede no darse cuenta (o no saber) que devuelve un 0 cuando el número que recibe como parámetro es menor que 0. Esto puede ocasionar errores en otras partes del programa y eso no es deseable.

Lo suyo sería indicar, de alguna manera, que esta función devuelve el factorial correctamente si no es llamada con un número negativo. Pero esto entra dentro del terreno de la semántica (el significado de los programas) y eso no se puede escribir mediante código.

1.1.4. Diseño por contrato

A la hora de programar es muy común que al ejecutar un programa este funcione correctamente sin que se produzcan errores durante la ejecución... pero el resultado no es el que nosotros esperábamos. Ese tipo de error se denomina **Error Lógico** y la gran mayoría de errores en los programas son de este tipo.

Un error lógico se produce porque hay una discrepancia entre lo que el programador quiere que haga el programa y lo que ha dicho que haga el programa. Pongamos un ejemplo muy sencillo: quiero multiplicar dos números y utilizo el símbolo de suma en lugar de el de multiplicación. Resultado: el programa funciona bien, pero suma los números en lugar de multiplicarlos.

Estamos acostumbrados a que los entornos de desarrollo sean capaces de detectar y corregir los errores léxicos y sintácticos⁸, e incluso algunos errores semánticos⁹. Sin embargo, no pueden detectar los errores lógicos.

En nuestro ejemplo anterior, el entorno de desarrollo no puede detectar que eso es un error, porque *no sabe* cuál es la intención del programador. Es decir: no conoce la semántica que el programador desea darle al programa.

Si bien hay un consenso universal sobre cómo se deben expresar los niveles léxico y sintáctico de los lenguajes de programación, lo que permite que se pueda automatizar la detección de errores a estos niveles, el nivel semántico se escapa de cualquier consenso y no existe una manera universalmente aceptada de expresar la semántica que se le desea dar a un programa.

En 1969, Hoare introdujo el **Diseño por Contrato** [1] en el que la semántica de los programas se establece mediante una serie de predicados lógicos que indican las propiedades que se cumplen en un punto concreto del programa.

En esta asignatura utilizaremos una versión del diseño por contrato algo menos estricta que la propuesta por Hoare y nos limitaremos a definir la semántica de los subprogramas.

En primer lugar vamos a introducir el concepto de **Precondición** en la Definición 1.

Definición 1. Una *precondición* es un predicado lógico que establece las condiciones que deben cumplir los parámetros de entrada de un subprograma *antes de su ejecución*.

⁸Debidos, respectivamente (y sin entrar en demasiados detalles) al uso de caracteres no esperados o a una combinación incorrecta de instrucciones.

⁹Como errores de tipo, divisiones por 0 y otros de este estilo.

En caso de que no sea necesaria ninguna condición sobre los parámetros de entrada, la precondition tomará el valor **Verdadero**. Con esto indicamos que se cumple siempre, con independencia de los valores de los parámetros de entrada.

Ahora podemos reescribir el nuestro algoritmo que calcula el factorial (algoritmo 4) introduciendo una precondition, tal y como hacemos en el algoritmo 7.

Algoritmo 7 Pseudocódigo factorial de un número, con precondition.

Precondición: $n \geq 0$

```

1: Función FACTORIAL(entero  $n$ )
2:   Si  $n == 0$  entonces
3:     devolver 1
4:   Si no
5:     devolver  $n * \text{factorial}(n - 1)$ 
6:   Fin Si
7: Fin Función

```

Como se puede ver, la precondition no forma parte del código del subprograma y debe entenderse como una documentación que el programador proporciona al usuario del subprograma. ¿Qué significado debemos darle a una precondition?

Si el usuario llama al subprograma incumpliendo la precondition, el programador del subprograma no se hace responsable de los posibles errores que se produzcan.

Es decir, la responsabilidad de realizar llamadas de forma correcta recae en quien realiza la llamada (el usuario del subprograma) y no en el programador. Esto tiene una implicación directa: **el código del subprograma no debe comprobar si se cumple o no la precondition**, directamente asumirá que se cumple.

Pero esto no nos basta, de momento el programador se está quitando responsabilidades, pero deberá asegurar que el código que ha realizado cumple correctamente con su cometido, si se cumplen las condiciones esperadas por el código.

Para ello, introducimos un segundo concepto en la definición 2: la postcondición.

Definición 2. Una **postcondición** es un predicado lógico que establece las condiciones que se deben cumplir **al finalizar la ejecución de un subprograma**.

Juntas, precondition y postcondición componen la **especificación** del subprograma. La relación entre precondition y postcondición se establece como sigue:

Si el usuario se compromete a llamar al subprograma cumpliendo la precondition, el programador del subprograma se compromete a que, al finalizar su ejecución, se alcancen las condiciones establecidas por la postcondición.

Así, la especificación de un subprograma es una suerte de contrato que firman programador y usuario y que incluye las dos cláusulas anteriores. Añadamos una postcondición a nuestro algoritmo de cálculo del factorial (ver algoritmo 8).

Algoritmo 8 Pseudocódigo factorial de un número, con precondition y postcondición.

Precondición: $n \geq 0$ **Postcondición:** $OUT = n!$

```

1: Función FACTORIAL(entero  $n$ )
2:   Si  $n == 0$  entonces
3:     devolver 1
4:   Si no
5:     devolver  $n * \text{factorial}(n - 1)$ 
6:   Fin Si
7: Fin Función

```

Como nuestro subprograma es una función, debe devolver un valor. Vamos a adoptar la convención de llamar *OUT* al valor devuelto por el subprograma. Así pues, la postcondición establece que el valor devuelto por la función **factorial** es, precisamente, el factorial del parámetro de entrada.

Gracias a este contrato es posible depurar más fácilmente los errores que se produzcan en nuestros programas:

- Si se incumple la precondition significa que al realizar la llamada no se ha comprobado que los valores de los parámetros cumplan la precondition. En nuestro ejemplo, esto se corresponde con una llamada a la función **factorial** en la que el parámetro contiene un valor negativo. Consecuencia: se produce un error Stack Overflow, pero la responsabilidad no es del programador del subprograma.
- Si se cumple la precondition, pero no se cumple la postcondición: el código del subprograma contiene un error. En nuestro ejemplo, esto sucedería si en la línea 5 hubiésemos utilizado el símbolo $+$ en lugar de $*$, provocando que el valor devuelto no fuese el establecido por la postcondición. Esto significaría que el programador del subprograma ha cometido un error en su código.

En algunas ocasiones, el subprograma no va a devolver un valor, sino que modificará alguno de los parámetros de entrada. En estos casos, si en la postcondición hacemos mención al parámetro de entrada que se modifica, ¿estamos referenciando su valor antes o después de la ejecución?

Para resolver esta ambigüedad, adoptaremos la siguiente convención: para distinguir entre el valor de un parámetro de entrada antes y después de la ejecución del subprograma, se añadirán los sufijos **IN** (para referirnos al valor antes de la ejecución) y **OUT** (para referirnos al valor después de la ejecución).

Vamos a poner como ejemplo el algoritmo 9 que recibe un array de n enteros y lo modifica duplicando cada uno de sus valores.

En la postcondición podemos ver que lo que se debe cumplir es que el valor de cada elemento i -ésimo del array **A**, para i de 1 a n , tras la ejecución del programa ($A_{out}[i]$) debe ser el doble que su valor antes de la ejecución del programa ($A_{in}[i]$).

Las especificaciones suponen una documentación muy útil para los usuarios de los subprogramas, ya que establecen las condiciones en las que se pueden utilizar e informan de las condiciones

Algoritmo 9 Duplicar los elementos de un array de enteros.

Precondición: Verdadero

Postcondición: $\forall i \in \{1, 2, \dots, n\} : A_{out}[i] = 2 \cdot A_{in}[i]$

```

1: Procedimiento DUPLICARARRAY(Array⟨entero⟩[m..n] A)
2:   Para i ← 1 hasta n hacer
3:     A[i] ← 2 · A[i]
4:   Fin Para
5: Fin Procedimiento

```

a las que se llega tras su ejecución.

1.2. Ejemplos de Algoritmos Numéricos

En esta sección vamos a ver algunos algoritmos recursivos sobre números aparte de los que ya hemos visto como el cálculo del factorial (algoritmo 4) o el de la Sucesión de Fibonacci (algoritmo 3).

Se propone como ejercicio clasificar cada uno de los algoritmos según los criterios establecidos (número de llamadas recursivas y forma de los casos recursivos) en la sección 1.1.2.

Algoritmo de Euclides

Este algoritmo se utiliza para calcular el máximo común divisor entre dos números enteros a y b . Se basa en el hecho de que el máximo común divisor entre a y b es igual al máximo común divisor entre b y el resto de la división de a entre b , es decir $a \bmod b$.

Algoritmo 10 Pseudocódigo con el algoritmo recursivo para calcular el máximo común divisor de dos enteros positivos según el Algoritmo de Euclides.

Precondición: $a > 0, b \geq 0$

Postcondición: $OUT = mcd(a, b)$

```

1: Función MCD(entero a, entero b)
2:   Si b == 0 entonces
3:     devolver a
4:   Si no
5:     devolver mcd(b, a mod b)
6:   Fin Si
7: Fin Función

```

Podemos ver que la precondición nos indica que este algoritmo va a funcionar si ambos parámetros son enteros positivos, en cuyo caso el valor devuelto será el máximo común divisor entre ellos.

Suma de los dígitos de un número

Para sumar los dígitos de un número simplemente deberemos separar el dígito de las unidades (que es el resto de dividir el número por 10) y añadirlo a la suma del resto de dígitos (que es

el resultado de la división entera del número por 10). Esto se repetirá hasta que el número sea inferior a 10, en cuyo caso la suma de su único dígito será él mismo.

Algoritmo 11 Pseudocódigo con el algoritmo recursivo para sumar los dígitos de un número.

Precondición: $n \geq 0$

Postcondición: OUT = suma de los dígitos del número n

```

1: Función SUMADIGITOS(entero  $n$ )
2:   Si  $n < 10$  entonces
3:     devolver  $n$ 
4:   Si no
5:     devolver  $n \bmod 10 + \text{sumaDigitos}(n \text{ div } 10)$ 
6:   Fin Si
7: Fin Función

```

Según la precondición del algoritmo 11, este algoritmo requiere que el número que le pasemos como parámetro a la función sea no negativo. Si es así, la función nos devolverá la suma de los dígitos del número recibido.

Multiplicación rusa

Este algoritmo se utiliza para multiplicar dos números basándonos únicamente en la suma y en la multiplicación y división por 2.

Se le conoce popularmente como “*multiplicación rusa*” pues en los siglos XVIII y XIX se empleó esta denominación al atribuir el uso de este algoritmo a los campesinos rusos que podían realizar multiplicaciones sin conocer completamente las tablas de multiplicar.

Sin embargo, su origen es muy anterior y se remonta, como mínimo, al Antiguo Egipto, pues se han encontrado menciones indirectas al mismo en el *Papiro de Rhind*[2], que ha sido datado alrededor del año 1550 a.C.

Algoritmo 12 Pseudocódigo con el algoritmo de la multiplicación rusa.

Precondición: $a \geq 0, b \geq 0$

Postcondición: OUT = $a * b$

```

1: Función MULTRUSA(entero  $a$ , entero  $b$ )
2:   Si  $b == 0$  entonces
3:     devolver 0
4:   Si no
5:     Si  $b$  es par entonces
6:       devolver  $\text{multRusa}(2*a, b \text{ div } 2)$ 
7:     Si no
8:       devolver  $a + \text{multRusa}(2*a, b \text{ div } 2)$ 
9:     Fin Si
10:  Fin Si
11: Fin Función

```

Como podemos ver, el algoritmo se dedica a duplicar uno de los parámetros y a reducir a la mitad el otro¹⁰, hasta llegar a 0. Si el parámetro que se reduce es impar, se suma al resultado

¹⁰Entendiendo *reducir a la mitad* como la división entera por defecto.

el primer parámetro, mientras que si es par, no se realiza ninguna otra operación.

Este algoritmo ha sido muy empleado en los ordenadores equipados con microprocesadores de 8 bits, ya que la gran mayoría de ellos no disponía de instrucciones nativas de multiplicación, pero sí de instrucciones de suma y de desplazamiento de bits a izquierda y derecha¹¹.

Evaluación de un polinomio (algoritmo de Horner)

Un **polinomio** es una expresión algebraica formada por una suma finita de términos, donde cada término es el producto de un coeficiente por una potencia entera no negativa de una variable. De forma general, un polinomio de grado n puede escribirse como

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 = \sum_{i=0}^n a_i x^i,$$

donde $a_0, a_1, \dots, a_n$ son los coeficientes del polinomio, $a_n \neq 0$ y n es su grado.

El algoritmo de Horner evalúa un polinomio de forma más eficiente que realizando explícitamente todas las operaciones indicadas¹². Su fundamento consiste en reescribir el polinomio mediante aplicaciones sucesivas de la propiedad distributiva de la multiplicación respecto de la suma, reduciendo así el número de operaciones necesarias para su evaluación.

Es decir, el polinomio

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

puede reescribirse como

$$P(x) = ((\cdots (a_n x + a_{n-1}) x + a_{n-2}) x + \cdots + a_1) x + a_0.$$

lo que nos lleva, directamente, al algoritmo 13.

Algoritmo 13 Pseudocódigo con el algoritmo de Horner para evaluar un polinomio.

Precondición: $P[n] \neq 0, 0 \leq i \leq n$

Postcondición: OUT = valor del polinomio $P_{[i \dots n]}$ en x .

```

1: Función HORNER(Array<real>[0..n] P, real x, entero i)
2:   Si i == n entonces
3:     devolver P[n]
4:   Si no
5:     devolver P[i] + x * horner(P,x,i + 1)
6:   Fin Si
7: Fin Función
```

¹¹Estas dos últimas, en binario, equivalen a multiplicar y dividir por 2 respectivamente.

¹²Es decir: calcular las potencias de x , multiplicar cada una de ellas por su coeficiente y sumar los términos resultantes.

Este algoritmo recibe un array P de números reales, indexados desde 0 hasta n , que representan los coeficientes del polinomio P , de grado n . Sobre dichos coeficientes se establece la precondición de que el coeficiente $P[n]$ ha de ser no nulo. También recibe un valor x real en el que se quiere evaluar el polinomio y un índice i que ha de estar entre 0 y n .

El resultado, según establece la postcondición, es el valor del polinomio $P_{[i..n]}$ en x . Dicho polinomio está formado únicamente por los coeficientes almacenados en las posiciones $P[i], \dots, P[n]$. Además, los exponentes asociados a cada coeficiente $P[j]$ con $i \leq j \leq n$ ya no están representados directamente por j , sino por $j - i$, de manera que el coeficiente $P[i]$ es el término independiente de dicho polinomio.

De esta manera, una llamada inicial como:

`horner(P,x,0)`

evalúa el polinomio completo P en el punto x , pues al comenzar con $i = 0$ el polinomio considerado es precisamente $P_{[0..n]} = P$.

1.3. Ejemplos de Estructuras de Datos

Hasta ahora, hemos considerado la recursión como una forma de programación que nos permite expresar muy fácilmente soluciones a problemas que puedan expresarse en función de una solución a una versión más simple de ellos mismos.

Sin embargo, la recursión va más allá, ya que también nos permite definir estructuras de datos de manera recursiva.

A modo de introducción de lo que veremos en los capítulos 4 y 5, vamos a presentar un par de ejemplos de dos estructuras del mundo real que pueden ser expresadas recursivamente.

1.3.1. La cola para pagar en el supermercado

Estamos comprando en nuestro supermercado favorito y ya hemos llenado el carrito, así que nos dirigimos a la caja para pagar nuestra compra. Llegamos y nos ponemos a la cola.

La cola de la caja puede ser vista como una estructura de datos secuencial: cada elemento tiene un único predecesor (salvo el primero) y un único sucesor (salvo el último). El cajero va procesando (en este caso, cobrando) cada uno de los elementos en orden.

De hecho, la estructura de datos secuencial que mejor representa la cola del supermercado se denomina, precisamente, cola¹³.

¹³Como veremos más adelante, una cola es una estructura secuencial con un punto de entrada y otro (diferente) de salida. Permite recorrer sus elementos en el mismo orden en el que entraron y, por tanto, la política que sigue se denomina FIFO (First In, First Out).

Además de su descripción secuencial, una cola también admite una definición recursiva:

- Si no hay nadie esperando, significa que la caja está vacía, o lo que es lo mismo, la cola de clientes para la caja es vacía.
- Si hay, al menos, un cliente en la cola, tras pagar el cliente saldrá de la caja y todos los que aún queden esperando formarán otra cola.

Es decir:

- Una cola vacía es una cola.
- Una cola formada por un primer elemento seguido de otra cola también es una cola.

Esta forma de definir una cola es un ejemplo de definición recursiva de una estructura de datos.

1.3.2. Las carpetas del ordenador

Estamos buscando un archivo en nuestro ordenador. Entramos en una carpeta y, si no lo encontramos, abrimos cada una de sus subcarpetas para seguir buscando. En cada una de ellas repetimos exactamente el mismo proceso.

El proceso que hemos descrito es, precisamente, un algoritmo recursivo: para cada una de las subcarpetas que nos encontremos, vamos a realizar el mismo proceso. Es decir, aplicaremos recursividad.

Esto es posible, porque una carpeta puede definirse recursivamente:

- Una carpeta puede contener solo archivos, pero ninguna subcarpeta.
- Una carpeta puede contener archivos y subcarpetas, cada una de las cuales será, a su vez, una carpeta.

O, lo que es lo mismo:

- Una carpeta sin subcarpetas es una carpeta.
- Una carpeta puede contener otras carpetas, cada una de las cuales es otra carpeta.

Así pues, la estructura de carpetas de nuestro ordenador puede definirse recursivamente como hemos hecho.

1.3.3. Definiciones recursivas de estructuras de datos

En base a los ejemplos anteriores, vemos un paralelismo entre las definiciones recursivas de estructuras de datos y las definiciones recursivas de algoritmos:

- Debe existir un caso base que nos defina la estructura más sencilla posible sin necesidad de recursión.
- Debe existir una regla de construcción que nos permita obtener estructuras más complejas a partir de otras de su misma naturaleza.

En los dos ejemplos anteriores:

- Una cola o bien es la cola vacía (*caso base*) o un elemento seguido de una cola (*regla de construcción*).
- Una carpeta o bien no contiene subcarpetas (*caso base*) o, si las contiene (*regla de construcción*), cada una de ellas es, a su vez, una carpeta.

En los próximos capítulos aparecerán otras estructuras, como las listas enlazadas, los árboles o los grafos, cuya naturaleza recursiva es aún más evidente y en las que esta forma de razonamiento permitirá diseñar algoritmos de manera sencilla y natural.

Capítulo 2

Introducción al Coste Computacional

El análisis del coste computacional estudia **cómo medir los recursos necesarios para ejecutar un algoritmo en función del tamaño de los datos de entrada**, de manera que se pueda comparar la eficiencia de distintas soluciones a un mismo problema. En concreto, estos recursos habitualmente son el tiempo y la cantidad de memoria:

- **Coste/Complejidad temporal:** se refiere al **número de operaciones básicas/elementales** (descritas más adelante) que realiza un algoritmo en función del tamaño de los datos de entrada. A partir de ese cálculo se puede estimar el tiempo de ejecución del algoritmo o, más exactamente, **se obtiene una aproximación del crecimiento del tiempo de ejecución según aumenta el tamaño de los datos de entrada**. Se trata del análisis computacional más habitual que **abordaremos en esta asignatura**.
- **Coste/Complejidad espacial:** se refiere a la cantidad de memoria (número de celdas de memoria) que necesita un algoritmo durante su ejecución en función del tamaño de los datos de entrada. Este análisis es especialmente relevante cuando los algoritmos analizados crean copias de datos, emplean estructuras de datos auxiliares o utilizan recursividad. No obstante, este aspecto **no se abordará en esta asignatura**.

En este tema se presenta brevemente el **Modelo RAM** como marco teórico para determinar cuáles operaciones se consideran básicas o elementales. A continuación, se introduce el concepto de **función de coste temporal de un algoritmo** y los criterios más habituales para calcularla. Posteriormente, se expone la **notación de Landau** para expresar órdenes de crecimiento del coste computacional en función del tamaño de los datos de entrada, junto con la **jerarquía de costes** para comparar la eficiencia de distintos algoritmos. Por último, se ofrece un estudio introductorio sobre el **cálculo del coste temporal de algoritmos iterativos y recursivos**. Los contenidos de este capítulo permitirán evaluar la eficiencia de las principales operaciones de las estructuras de datos tratadas en los capítulos posteriores.

2.1. Modelo RAM

El análisis del coste computacional requiere un modelo que permita calcular el **número de operaciones elementales realizadas por un algoritmo**. En esta asignatura utilizaremos el **Modelo RAM** (*Random Access Machine*¹), que constituye un marco teórico habitual para el estudio introductorio de la eficiencia de algoritmos y estructuras de datos. El Modelo RAM simplifica el escenario real (por ej. no considera latencias de memoria, cachés, paralelismo,

¹No confundir con Memoria RAM (*Random Access Memory*).

etc.), pero proporciona una aproximación suficientemente precisa para comparar algoritmos y estructuras de datos de forma independiente del *hardware* que se adecúa con los objetivos de esta asignatura.

2.1.1. Características del Modelo RAM

El Modelo RAM asume una máquina ideal con las siguientes **características** e hipótesis:

- **Memoria de acceso aleatorio (Memoria RAM, *Random Access Memory*):** el Modelo RAM supone una memoria ilimitada (o suficientemente grande) formada por un conjunto de celdas numeradas $M[0], M[1], M[2], \dots$, de modo que las direcciones de memoria son los índices de M , es decir, números enteros no negativos $0, 1, 2, \dots$. También se asume *acceso aleatorio*: se puede acceder (lectura y escritura) a cualquier posición de la memoria $M[i]$ en tiempo constante con independencia de cuál sea la posición i y su distancia con respecto a la celda a la que se accedió con anterioridad. Cada celda de la memoria es capaz de almacenar una cierta cantidad de bits denominada *palabra*, de la que hablaremos en el siguiente punto.
- **Tamaño de palabra suficientemente grande:** en nuestro contexto, podemos asumir que el *tamaño de palabra* en una memoria RAM es la cantidad de bits (*palabra*) que la memoria puede transferir en una única operación. El Modelo RAM supone que los datos elementales (enteros, *strings*, etc.) siempre caben en una palabra y muchas de sus operaciones elementales se realizan en tiempo constante.

Los ordenadores actuales suelen tener un tamaño de palabra de 32 bits o, más comúnmente, 64 bits. El *Modelo Word-RAM* es una variante del modelo RAM que asume un tamaño de palabra w , pero se trata de un marco teórico más adecuado para escenarios no abordados en esta asignatura, por ej. algoritmos sobre números grandes, algoritmos de bajo nivel con operaciones bit a bit (*bitwise*), etc.

- **Tiempo uniforme por operación elemental:** el modelo RAM supone que cada operación elemental tiene un coste acotado por una constante que normalmente se toma por convenio como 1 unidad de tiempo (u.t.), por ej. 1 nanosegundo (ns). En un escenario real, algunas operaciones elementales pueden suponer mayor tiempo que otras², pero dicha diferencia puede considerarse despreciable y el modelo RAM simplemente la abstrae.
- **Ejecución secuencial:** se asume que las instrucciones se ejecutan una tras otra, sin paralelismo.

2.1.2. Operaciones elementales del Modelo RAM

Las **operaciones elementales o básicas** son instrucciones sencillas de programación caracterizadas por ser **independientes del tamaño de sus datos de entrada** y para las que se asume que se ejecutan con **coste constante**: 1 u.t. bajo la asunción del tiempo uniforme del

²O incluso ocurre con una misma operación, por ej. acceder a una u otra posición de memoria.

modelo RAM. En concreto, en este modelo se consideran elementales/básicas a las siguientes operaciones:

- **Acceso a memoria:** lectura o escritura de una posición de memoria.
- **Asignación de variables de tipos básicos.**
- **Operaciones aritméticas básicas:** suma, resta, multiplicación, división³, división entera (div, // en Python), resto (mod, % en Python), etc.
- **Operaciones lógicas/booleanas:** NOT ($\neg$), AND ($\wedge$), OR ($\vee$), XOR ($\oplus$), etc.
- **Comparación entre valores básicos:** =, $\neq$, <, $\leq$, >, $\geq$, etc.
- **Acceso a una componente de un array de tipos básicos:** el modelo RAM asume que un array a es un bloque contiguo de memoria, de manera que el acceso a una componente $a[i]$ se modela como un acceso a memoria $M[dir(a) + i]$, donde $M[k]$ accede a la k -ésima celda de memoria y $dir(a)$ es la dirección de memoria de la primera componente del array a ⁴. Nótese que dicho acceso puede descomponerse en dos operaciones elementales: una suma y un acceso a memoria, pero en global es **independiente del tamaño** del array a . No obstante, el Modelo RAM abstrae estos detalles internos y lo asume como una única operación elemental con un coste de una unidad de tiempo.
- **Sentencia de retorno:** return.
- **Sentencias de Entrada/Salida (E/S):** print o similares.

Nótese que si se asumiese un tamaño de palabra fijo (por ej. Modelo Word-RAM con $w = 32$ bits o $w = 64$ bits), salvo un acceso a una posición de memoria, las anteriores operaciones tendrían coste constante si los operandos caben en una palabra (su tamaño es $\leq w$), mientras que el coste no sería constante si algún operando ocupa varias palabras (su tamaño es $> w$), de modo que las operaciones solo entonces sí **dependen del tamaño de los datos**: en concreto, dependen del número de palabras que ocupan los operandos/parámetros.

2.1.3. Cálculo del coste temporal en el Modelo RAM

Como se dijo anteriormente, el análisis del coste computacional temporal se refiere al **número de operaciones elementales que realiza un algoritmo**. Tomemos como ejemplo el Algoritmo 9 visto en el Capítulo 1 que duplica el valor de todas las componentes de un array de enteros. En este ejemplo, el número de operaciones elementales realizadas sí **depende del tamaño** del array ya que su coste crece proporcionalmente al número de componentes del array.

A continuación, se detalla el conteo de operaciones elementales realizadas por el algoritmo línea a línea, obviando las líneas 1, 4 y 5 porque son declaraciones de inicio o fin de procedimiento o bucle, irrelevantes en términos de coste computacional:

³En escenarios de análisis más finos, algunas operaciones como la división no se consideran básicas.

⁴Por las asunciones del modelo sobre la memoria RAM y el tamaño de palabra, cada componente del array ocupa una única celda de memoria. Si no fuera así, entonces el acceso a memoria sería $M[dir(a) + i * s]$, siendo s el tamaño del dato.

Línea 2: **Para** $i \leftarrow 1$ **hasta** n **hacer**. Recorrido del array.

Salvo en ciertos casos muy específicos, los bucles **for** equivalen a los bucles **while** porque expresan las mismas instrucciones de manera más cómoda y compacta al precio de ocultar algunas operaciones (importante en nuestro contexto). En jerga informática, se dice que los bucles **for** son un *azúcar sintáctico* de los bucles **while**. El Algoritmo 14 muestra la equivalencia general entre ambos tipos de bucles y un ejemplo bastante habitual en donde se usa una variable de control de tipo entero i que recorre un rango de valores (de 1 hasta n) y cuya actualización consiste en incrementarse en una unidad. En ambos casos, la línea 1 del bucle **for** equivale a las líneas 1 y 2 del bucle **while**; la línea 2 del bucle **for** equivale a las líneas 3 y 4 del bucle **while**; y la línea 3 del bucle **for** equivale a la línea 5 del bucle **while** (irrelevantes con respecto al coste computacional).

Algoritmo 14 Equivalencia entre bucles **for** y bucles **while**

Equivalencia general:

1: Para inicialización; condición; actualización hacer	$\iff$	1: inicialización
2: cuerpo		2: Mientras condición hacer
3: Fin Para		3: cuerpo
		4: actualización
		5: Fin Mientras

Ejemplo habitual:

1: Para $i \leftarrow 1$ hasta n hacer	$\iff$	1: $i \leftarrow 1$
2: cuerpo		2: Mientras $i \leq n$ hacer
3: Fin Para		3: cuerpo
		4: $i \leftarrow i + 1$
		5: Fin Mientras

Volviendo al ejemplo del Algoritmo 9, aplicando esta equivalencia, la línea 3 consiste en las siguientes operaciones:

- Una asignación $i \leftarrow 1$, 1 op. elemental.
- Comparaciones $i \leq n$. Cada comparación es una operación elemental, de modo que hay que razonar cuántas comparaciones se realizan sabiendo que inicialmente $i = 1$, i se incrementa en 1 en cada iteración y la condición deja de cumplirse cuando $i = n + 1$. Por tanto, se realizan $n + 1$ comparaciones, de manera que en n de ellas (con $i = 1, i = 2, \dots, i = n$) se entra en el bucle y en una de ellas ($i = n + 1$) se sale del bucle al incumplirse la condición. Por tanto, se realizan $n + 1$ ops. elementales.
- **TOTAL:** $1 + (n + 1) = n + 2$ **operaciones elementales**

Línea 3: $A[i] \leftarrow 2 \cdot A[i]$. Cuerpo del bucle: duplicar componentes del array.

- Como se razonó anteriormente, el cuerpo del bucle se repite en n ocasiones, de modo que el número de operaciones internas del cuerpo del bucle se multiplicará por n .

- Lectura de $A[i]$, 1 op. elemental.
- Multiplicación $2 \cdot A[i]$, 1 op. elemental.
- Asignación $A[i] \leftarrow 2 \cdot A[i]$, 1 op. elemental.
- Actualización índice i : $i \leftarrow i + 1$, una suma y una asignación, 2 ops. elementales.
- **TOTAL:** $n \cdot (1 + 1 + 1 + 2) = 5 \cdot n$ operaciones elementales

Tras sumar todo lo anterior, tenemos que el algoritmo $\mathcal{A} = \text{duplicarArray}$ realiza un total de $T_{\mathcal{A}}(n) = (n + 2) + (5 \cdot n) = 6 \cdot n + 2$ operaciones elementales, siendo n el tamaño del array que recibe como entrada. Dado un cierto tamaño de entrada n , podemos dar una estimación del tiempo que tarda el algoritmo bajo la asunción del Modelo RAM de que 1 op. elemental tarda 1 u.t. (ver apartado 2.1.1).

2.2. Función de coste temporal de un algoritmo

El ejemplo anterior ilustra cómo, dentro del Modelo RAM, es posible determinar el coste temporal de un algoritmo contando el número de operaciones elementales que ejecuta (o, equivalentemente, en u.t.) y expresándolo mediante una función. En este apartado se define de manera formal el concepto de **función de coste temporal de un algoritmo** y los criterios más comunes utilizados para su cálculo.

En lo sucesivo, usaremos las siguientes notaciones para referirnos a varios conjuntos de números no negativos⁵:

- **Números naturales ($\mathbb{N}$).** La inclusión del cero en el conjunto de los números naturales depende del contexto o de la convención adoptada. Para evitar ambigüedades, trabajaremos con los siguientes conjuntos:
 - **Enteros no negativos:** $\mathbb{N}_0 = \{0, 1, 2, \dots\}$.
 - **Enteros positivos:** $\mathbb{N}^+ = \mathbb{N}_0 \setminus \{0\} = \{1, 2, 3, \dots\}$.
- **Números reales ($\mathbb{R}$).** Análogamente, consideraremos los siguientes subconjuntos de los números reales:
 - **Reales no negativos:** $\mathbb{R}_{\geq 0} = \{x \in \mathbb{R} \mid x \geq 0\}$.
 - **Reales positivos:** $\mathbb{R}^+ = \mathbb{R}_{\geq 0} \setminus \{0\} = \{x \in \mathbb{R} \mid x > 0\}$.

Por otro lado, como se ha ido comentando con anterioridad, el coste computacional de un algoritmo se expresa como una función que **depende del tamaño de sus datos de entrada**. Por ello, requeriremos los siguientes **conceptos y notaciones sobre las entradas de un algoritmo**:

⁵Excluimos números negativos por no tener sentido en nuestro contexto al referirnos al n° de ops. elementales/u.t. (ídem para el caso de la cantidad de memoria), tamaño de entradas de datos, etc.

- Dado un algoritmo $\mathcal{A}$, denotaremos por $I^{\mathcal{A}}$ al conjunto de todas sus entradas (*inputs*).
- La **función tamaño de entrada** $|\cdot|^{\mathcal{A}} : I^{\mathcal{A}} \rightarrow \mathbb{N}_0$ asigna a cada entrada $x \in I^{\mathcal{A}}$ su tamaño $|x|^{\mathcal{A}}$ expresado como un entero no negativo ⁶.
- Denotaremos por $I_n^{\mathcal{A}} = \{x \in I^{\mathcal{A}} \mid |x|^{\mathcal{A}} = n\}$ al conjunto de las entradas del algoritmo $\mathcal{A}$ con tamaño $n \in \mathbb{N}_0$.
- La **función de coste sobre entradas** $t_{\mathcal{A}} : I^{\mathcal{A}} \rightarrow \mathbb{N}_0$ asigna el número total de ops. elementales ejecutadas por $\mathcal{A}$ al recibir una cierta entrada. Es decir, $t_{\mathcal{A}}(x)$ es el número total de ops. elementales ejecutadas por el algoritmo $\mathcal{A}$ recibiendo la entrada x .

Con los anteriores ingredientes, la definición 3 presenta de manera precisa qué se entiende como **función de coste temporal de un algoritmo**.

Definición 3. La **función de coste temporal de un algoritmo $\mathcal{A}$** es una función $T_{\mathcal{A}} : \mathbb{N}_0 \rightarrow \mathbb{R}_{\geq 0}$ con la forma:

$$T_{\mathcal{A}}(n) = \bigodot_{x \in I_n^{\mathcal{A}}} t_{\mathcal{A}}(x)$$

donde $\bigodot$ es un operador matemático que depende del criterio con el que se mida el coste. Los criterios más habituales son los siguientes:

- Si $\bigodot = \text{máx}$, obtenemos el **coste en el caso peor**:

$$T_{\mathcal{A}}^{\text{peor}}(n) = \text{máx}_{x \in I_n^{\mathcal{A}}} \{t_{\mathcal{A}}(x)\}$$

- Si $\bigodot = \text{mín}$, obtenemos el **coste en el caso mejor**:

$$T_{\mathcal{A}}^{\text{mejor}}(n) = \text{mín}_{x \in I_n^{\mathcal{A}}} \{t_{\mathcal{A}}(x)\}$$

- En el **coste en el caso promedio**, $\bigodot$ se corresponde con el valor esperado (esperanza matemática) con respecto a una distribución de probabilidad $\mathbb{P}$ sobre $I_n^{\mathcal{A}}$, i.e.:

$$T_{\mathcal{A}}^{\text{prom}}(n) = \mathbb{E}_{x \in I_n^{\mathcal{A}}} [t_{\mathcal{A}}(x)] = \sum_{x \in I_n^{\mathcal{A}}} t_{\mathcal{A}}(x) \cdot \mathbb{P}(x),$$

donde $\mathbb{P}(x)$ es la probabilidad de que x sea la entrada del algoritmo^a.

^aPor ej. si se asume que todas las entradas son equiprobables: $\forall x \in I_n^{\mathcal{A}} : \mathbb{P}(x) = \frac{1}{|I_n^{\mathcal{A}}|}$.

Para calcular el **coste temporal en el caso peor** hay que **identificar las entradas del algoritmo que implican el mayor número de operaciones**. Análogamente, para calcular el **coste temporal en el caso mejor** hay que **identificar las entradas que implican el menor número de operaciones**. Por último, para calcular el **coste temporal en el caso promedio**, hay que analizar el comportamiento del algoritmo considerando todas las posibles

⁶Cuando hay estructuras de datos involucradas, el tamaño suele coincidir con su número de elementos, como en el ejemplo del Algoritmo 9.

entradas y calcular el número medio de operaciones, asumiendo una distribución de probabilidad sobre dichas entradas.

El análisis del *coste es idéntico para los tres criterios cuando el funcionamiento del algoritmo es independiente de los valores de la entrada*. Esto sucede con el ejemplo del Algoritmo 9: se limita a recorrer el array duplicando el valor de cada componente con independencia de los valores contenidos en el array. Sin embargo, en muchos casos, esto no es así. El Algoritmo 15 se trata de la *búsqueda lineal* sobre un array: se recibe como entrada un array junto a un valor y el algoritmo determina si alguna componente del array contiene dicho valor. En este caso, el funcionamiento del algoritmo depende de los valores de la entrada, en particular si alguna componente del array contiene el valor buscado.

Algoritmo 15 Búsqueda lineal

Precondición: Verdadero

Postcondición: $\text{OUT} = \exists i \in \{1, 2, \dots, n\} : A[i] = v$

```

1: Función BUSQUEDALINEAL(Array⟨entero⟩[1..n] A, entero v)
2:   Para  $i \leftarrow 1$  hasta  $n$  hacer
3:     Si  $A[i] = v$  entonces devolver verdadero
4:   Fin Si
5: Fin Para
6:   devolver falso
7: Fin Función
  
```

Como el Algoritmo 15 se emplea con valores de tipo básico (enteros), las operaciones básicas (asignaciones, accesos, comparaciones...) son análogas a las vistas en el ejemplo del Algoritmo 9. El retorno del valor, que no estaba presente en el anterior ejemplo, solo añade una operación adicional. Entonces, el coste del algoritmo varía únicamente según el número de comparaciones $A[i] = v$. Asumiendo como función de coste del algoritmo dicho número de comparaciones, vemos lo siguiente: el coste $t_A(x)$ de una cierta entrada x viene acotado por el número de veces que se ejecuta el bucle **for**. Para aquellas entradas en las que el array A contiene el valor v , el número de comparaciones depende de cuál sea la primera posición del array en la que aparece el valor v . Dicha posición coincide con el número de comparaciones realizadas por el algoritmo. Por otro lado, para aquellas entradas en las que el array A no contiene el valor v , se realizan n comparaciones, se sale del bucle y finalmente se retorna **falso**. Por tanto, la función de coste sobre entradas es:

$$t_A(x) = \begin{cases} i & \text{si } A[i] = v \wedge \nexists j < i : A[j] = v \\ n & \text{si } \nexists i \in \{1, 2, \dots, n\} : A[i] = v \end{cases}$$

de manera que $t_A(x) \in \{1, 2, \dots, n\}$ según cuál sea la primera posición de A donde aparece v o n si A no contiene el valor v . Por tanto, ya sabemos cuáles son los posibles costes $t_A(x)$ para cualquier entrada del algoritmo.

A continuación, realizamos el análisis del coste mediante los diferentes criterios:

- **Caso peor:** el número máximo de comparaciones sucede cuando todas las componentes del array (n elementos) se comparan con el valor v . Esto ocurre cuando v se encuentra

en la última posición, i.e. $A[n] = v$, o bien cuando v no se encuentra en el array. En ambos casos se realizan n comparaciones. No obstante, simplemente aplicando la fórmula se tiene: $T_{\mathcal{A}}^{peor}(n) = \max_{x \in I_n^{\mathcal{A}}} \{t_{\mathcal{A}}(x)\} = \max\{1, 2, \dots, n\} = n$.

- **Caso mejor:** el número mínimo de comparaciones sucede para las entradas que cumplen $A[1] = v$ porque tras dicha comparación inmediatamente se retorna **verdadero**. No obstante, simplemente aplicando la fórmula se tiene: $T_{\mathcal{A}}^{mejor}(n) = \min_{x \in I_n^{\mathcal{A}}} \{t_{\mathcal{A}}(x)\} = \min\{1, 2, \dots, n\} = 1$.
- **Caso promedio:** para este caso es necesario contar con una cierta distribución de probabilidad. Al tratarse de un ejemplo ilustrativo, haremos las siguientes suposiciones al respecto:
 - Suponemos que v está en el array A con una cierta probabilidad $p \in [0, 1]$ que denotaremos como $\mathbb{P}(v \in A) = p$, de manera que si v no está en A , la probabilidad es $\mathbb{P}(v \notin A) = 1 - p$, al no haber otros casos posibles.
 - Si $v \in A$, suponemos equiprobabilidad sobre la primera posición del array en la que aparece el valor v : $\forall i \in \{1, 2, \dots, n\} : \mathbb{P}(A[i] = v) = \frac{1}{n}$.

Nótese que la anterior suposición sobre la distribución de todo el conjunto de entradas $I_n^{\mathcal{A}}$ se abstrae sobre propiedades de dichas entradas y dichas propiedades pueden verse como variables aleatorias, siempre y cuando se cubra todo el dominio de las entradas $I_n^{\mathcal{A}}$. En este caso, esto se consigue considerando estas dos propiedades vistas como variables aleatorias: (i) A contiene/no contiene el valor v ; y (ii) en caso afirmativo, cuál es la primera posición de A en la que aparece v .

Aplicando la fórmula para $T_{\mathcal{A}}^{prom}$ se tiene:

$$\begin{aligned} T_{\mathcal{A}}^{prom}(n) &= \mathbb{E}_{x \in I_n^{\mathcal{A}}} [t_{\mathcal{A}}(x)] = \mathbb{P}(v \in A) \cdot \mathbb{E}[t_{\mathcal{A}}(x) \mid v \in A] + \mathbb{P}(v \notin A) \cdot \mathbb{E}[t_{\mathcal{A}}(x) \mid v \notin A] = \\ &= p \cdot \mathbb{E}[t_{\mathcal{A}}(x) \mid v \in A] + (1 - p) \cdot \mathbb{E}[t_{\mathcal{A}}(x) \mid v \notin A] \end{aligned}$$

Para el caso de $\mathbb{E}[t_{\mathcal{A}}(x) \mid v \notin A]$, en todas esas entradas siempre se realizan $t_{\mathcal{A}}(x) = n$ comparaciones, por tanto el valor esperado es $\mathbb{E}[t_{\mathcal{A}}(x) \mid v \notin A] = n$.

Para el caso de $\mathbb{E}[t_{\mathcal{A}}(x) \mid v \in A]$, las entradas realizarán un número de comparaciones $t_{\mathcal{A}}(x) = i \in \{1, 2, \dots, n\}$ según cuál sea la primera posición de A donde aparece v . Con esto, podemos realizar el siguiente cálculo:

$$\mathbb{E}[t_{\mathcal{A}}(x) \mid v \in A] = \sum_{i=1}^n t_{\mathcal{A}}(x) \cdot \mathbb{P}(A[i] = v) = \sum_{i=1}^n i \cdot \frac{1}{n} = \frac{1}{n} \cdot \sum_{i=1}^n i = \frac{1}{n} \cdot \frac{n \cdot (n+1)}{2} = \frac{n+1}{2}$$

donde se ha usado la igualdad $\sum_{i=1}^n i = \frac{n \cdot (n+1)}{2}$, que puede demostrarse por inducción sobre $n \in \mathbb{N}^+$.

Por tanto:

$$T_{\mathcal{A}}^{prom}(n) = p \cdot \left(\frac{n+1}{2}\right) + (1 - p) \cdot n$$

de manera que si $v \in A$ (i.e., $p = 1$) entonces $T_A^{prom}(n) = \frac{n+1}{2}$, y si $v \notin A$ (i.e., $p = 0$) entonces $T_A^{prom}(n) = n$ (como ya razonamos en el caso peor). Es decir: el coste es menor cuanto más probable sea que $v \in A$, i.e. mayor sea el valor de p .

En esta asignatura **nos centraremos en el análisis del coste en el caso peor** porque garantiza una cota superior del coste computacional, es decir: el algoritmo $\mathcal{A}$ no tendrá mayor coste que el expresado por $T_A^{peor}(n)$. Esto proporciona una herramienta valiosa para comparar diferentes algoritmos que resuelven un mismo problema con independencia de la distribución de los datos de entrada y sin necesidad de asumir hipótesis a ese respecto. Sin embargo, haremos notar lo siguiente sobre los otros criterios:

- El análisis en el **caso mejor** es útil para identificar en qué condiciones un algoritmo tiene un rendimiento óptimo. Normalmente, dichas condiciones son dependientes de las entradas. Si tenemos certeza de que las entradas cumplen dichas condiciones, entonces tenemos certeza de que la elección de ese algoritmo es adecuada. Sin embargo, **esta situación ocurre muy pocas veces en escenarios reales**.
- El análisis en el **caso promedio** proporciona una descripción más realista del comportamiento de un algoritmo siempre y cuando contemos con una distribución de las entradas bien modelada. Por otro lado, tiene la desventaja de que **no siempre es sencillo de calcular**.

2.3. Notación de Landau

En los apartados anteriores se ha expuesto cómo determinar el coste temporal de un algoritmo contando el número de operaciones elementales que ejecuta y expresándolo mediante una función de coste teniendo en cuenta un cierto criterio sobre las entradas. Sin embargo, cuando se trabaja con algoritmos de cierta complejidad, el número de operaciones puede depender de múltiples factores (condiciones en bloques **if-then-else**, bucles anidados, llamadas recursivas, etc.), de manera que este enfoque resulta poco manejable. En análisis de algoritmos normalmente interesa **el orden de crecimiento del coste a medida que aumenta el tamaño de los datos de entrada**.

En el ejemplo del algoritmo 9 (**duplicarArray**) se obtuvo una función de coste $T_{\mathcal{A}}(n) = 6 \cdot n + 2$. Cuanto mayor sea el valor de n , el peso sobre el coste total $T_{\mathcal{A}}(n)$ del término constante 2 es cada vez más irrelevante. Por otro lado, el factor multiplicativo 6 se trata de un valor constante que no modifica la tasa de crecimiento con respecto al tamaño de la entrada n , simplemente cambia la escala. Estos valores constantes varían por factores circunstanciales, por ej. la máquina empleada, el compilador, el lenguaje de programación, etc. Por tanto, dichos valores pueden ignorarse a la hora de captar el orden de crecimiento del coste de un algoritmo. En este ejemplo, diremos que el orden de crecimiento del coste es lineal con respecto al tamaño de entrada n porque se expresa mediante una función lineal sobre n , i.e. un polinomio de primer grado sobre n . En general, diremos que un *algoritmo lineal* es aquel que tiene un *orden de crecimiento lineal* con respecto al tamaño de entrada n , es decir: su función de coste es de la forma $T_{\mathcal{A}}(n) = c_1 \cdot n + c_0$ (siendo $c_1 > 0$ y $c_0 \geq 0$ valores constantes).

Pongamos otro ejemplo: $T_A(n) = n^2 + 20 \cdot n + 1500$. Se trata de una función de coste expresada como un polinomio cuadrático (de segundo grado) con tres términos: término cuadrático (n^2), término lineal ($20 \cdot n$) y término constante (1500). La Tabla 2.1 muestra cómo cada término de esta función de coste contribuye al total $T_A(n)$ para diferentes valores de n , medido en ops. elementales (o u.t.).

n	n^2	$20n$	1500	$T_A(n)$
10	100 (5.56 %)	200 (11.11 %)	1500 (83.33 %)	1800 (100 %)
100	10000 (74.07 %)	2000 (14.81 %)	1500 (11.11 %)	13500 (100 %)
1000	1000000 (97.88 %)	20000 (1.96 %)	1500 (0.15 %)	1021500 (100 %)

Tabla 2.1: Contribución de cada término en $T_A(n) = n^2 + 20 \cdot n + 1500$ (valores y porcentajes).

Podemos observar dos aspectos: (i) solamente para valores pequeños de n , los términos lineal y constante contribuyen más al coste total; (ii) a medida que crece el valor del tamaño de entrada n , el término dominante es claramente el cuadrático n^2 . Aunque el término lineal tuviese un factor multiplicativo más alto, siempre habrá un tamaño de entrada $n = n_0$ a partir del cual pasaría a ser un término despreciable en términos de contribución al coste total. En general, diremos que un *algoritmo cuadrático* es aquel que tiene un *orden de crecimiento cuadrático* con respecto al tamaño de entrada n , es decir: su función de coste es de la forma $T_A(n) = c_2 \cdot n^2 + c_1 \cdot n + c_0$ (siendo $c_2 > 0$, $c_1 \geq 0$ y $c_0 \geq 0$ valores constantes).

Los anteriores ejemplos ilustran que nos interesará tener una **clasificación de los algoritmos según sea el orden de crecimiento de sus funciones de coste**, por ej. lineales, cuadráticos, etc. Esta clasificación nos permitirá establecer comparaciones sobre la *eficiencia* entre distintos algoritmos que resuelven el mismo problema: por ej. un algoritmo lineal será más eficiente que un algoritmo cuadrático para valores grandes de n , independientemente de los factores constantes de sus funciones de coste.

Este apartado explica la **notación de Landau** que proporciona una herramienta matemática adecuada para describir el orden de crecimiento de funciones de coste de algoritmos y que es ampliamente utilizada en el estudio del coste computacional. Además, se presenta la **jerarquía de órdenes de complejidad**, que permite clasificar algoritmos según su coste computacional.

2.3.1. Definición de la notación de Landau

La **notación de Landau**⁷ se usa para expresar el **coste asintótico temporal de un algoritmo**, es decir: cómo crece su tiempo de ejecución a medida que aumenta el tamaño de la entrada. Para ello, se definen conjuntos de funciones que comparten el mismo comportamiento asintótico, ignorando constantes y términos de menor orden. En particular, las siguientes definiciones introducen los conjuntos de funciones $\mathcal{O}(\cdot)$, $\Omega(\cdot)$ y $\Theta(\cdot)$ ⁸ que permiten determinar si una función f es una **cota superior asintótica** de otra función g (notación Big- $\mathcal{O}$, definición

⁷Toma el nombre del matemático alemán Edmund Landau (1877-1938).

⁸Letras griegas mayúsculas de *omicron*, *omega* y *theta*, respectivamente. Esta notación fue popularizada por el informático estadounidense Donald E. Knuth. La notación de Landau incluye las notaciones *little-o*, $o(\cdot)$, y *little-omega*, $\omega(\cdot)$, aunque no las abordaremos en esta asignatura.

4), si f es una **cota inferior asintótica** de otra función g (notación Big- Ω , definición 5), y si f es una **cota ajustada asintótica** de otra función g (notación Big- Θ , definición 6).

Definición 4 (Notación Big- $\mathcal{O}$). Sea $f : \mathbb{N}_0 \rightarrow \mathbb{R}_{\geq 0}$. Se define y denota el **conjunto de las funciones acotadas superiormente por f** como:

$$\mathcal{O}(f(n)) = \{g : \mathbb{N}_0 \rightarrow \mathbb{R}_{\geq 0} \mid \exists c_{sup} \in \mathbb{R}^+ \exists n_0 \in \mathbb{N}^+ : 0 \leq g(n) \leq c_{sup} \cdot f(n) \forall n \geq n_0\}$$

Se dice que $\mathcal{O}(f(n))$ es el **conjunto de funciones del orden de $f(n)$** y si $g \in \mathcal{O}(f)$ se dice que **g es del orden de f** . Además, se dice que $\mathcal{O}(f(n))$ define un **orden de complejidad**.

Definición 5 (Notación Big- Ω). Sea $f : \mathbb{N}_0 \rightarrow \mathbb{R}_{\geq 0}$. Se define y denota el **conjunto de las funciones acotadas inferiormente por f** como:

$$\Omega(f(n)) = \{g : \mathbb{N}_0 \rightarrow \mathbb{R}_{\geq 0} \mid \exists c_{inf} \in \mathbb{R}^+ \exists n_0 \in \mathbb{N}^+ : 0 \leq c_{inf} \cdot f(n) \leq g(n) \forall n \geq n_0\}$$

Definición 6 (Notación Big- Θ). Sea $f : \mathbb{N}_0 \rightarrow \mathbb{R}_{\geq 0}$. Se define y denota el **conjunto de las funciones acotadas ajustadamente por f** como:

$$\Theta(f(n)) = \{g : \mathbb{N}_0 \rightarrow \mathbb{R}_{\geq 0} \mid \exists c_{inf}, c_{sup} \in \mathbb{R}^+ \exists n_0 \in \mathbb{N}^+ : 0 \leq c_{inf} \cdot f(n) \leq g(n) \leq c_{sup} \cdot f(n) \forall n \geq n_0\}$$

i.e. funciones que están acotadas tanto inferiormente como superiormente por f , por lo que puede definirse de manera equivalente como:

$$\Theta(f(n)) = \mathcal{O}(f(n)) \cap \Omega(f(n))$$

El concepto de **coste asintótico temporal de un algoritmo** no debe confundirse con la función de coste de un algoritmo. Esta última se calcula contando ops. elementales (según el Modelo RAM en nuestro caso) una vez fijado un determinado criterio sobre las entradas (casos peor, mejor y promedio) como se explicó en los apartados 2.1.3 y 2.2.

Si $T_{\mathcal{A}}(n)$ es una función de coste de un algoritmo $\mathcal{A}$ (ya sea $T_{\mathcal{A}}^{peor}(n)$, $T_{\mathcal{A}}^{mejor}(n)$ o $T_{\mathcal{A}}^{prom}(n)$), la idea intuitiva de las anteriores definiciones de $\mathcal{O}(\cdot)$, $\Omega(\cdot)$ y $\Theta(\cdot)$ es la siguiente:

- $T_{\mathcal{A}}(n) \in \mathcal{O}(f(n))$: el tiempo de ejecución del algoritmo $\mathcal{A}$ **no crece más rápido que** una constante multiplicativa de la función $f(n)$ (*cota superior*) para tamaños de entrada n suficientemente grandes (iguales o superiores a un cierto valor n_0). Es decir: indica que **el tiempo de ejecución del algoritmo crece “como mucho” como $f(n)$** .
- $T_{\mathcal{A}}(n) \in \Omega(f(n))$: el tiempo de ejecución del algoritmo $\mathcal{A}$ **crece al menos tan rápido como** una constante multiplicativa de la función $f(n)$ (*cota inferior*) para tamaños de entrada n suficientemente grandes (iguales o superiores a un cierto valor n_0). Es decir: indica que **el tiempo de ejecución del algoritmo crece “como mínimo” como $f(n)$** .
- $T_{\mathcal{A}}(n) \in \Theta(f(n))$: el tiempo de ejecución del algoritmo $\mathcal{A}$ **crece al mismo orden que** la función $f(n)$, i.e.: está acotado superior e inferiormente por constantes multiplicativas de $f(n)$ para tamaños de entrada n suficientemente grandes (iguales o superiores a un

cierto valor n_0). Es decir: indica que **el tiempo de ejecución del algoritmo crece “de forma similar a” $f(n)$** .

Los conjuntos $\mathcal{O}(\cdot)$, $\Omega(\cdot)$ y $\Theta(\cdot)$ agrupan funciones con un comportamiento asintótico en común. En particular, $\Theta(\cdot)$ induce una *relación de equivalencia* (“ $g \approx f$ ”), mientras que $\mathcal{O}(\cdot)$ y $\Omega(\cdot)$ inducen *preórdenes*: cota superior (“ $g \leq f$ ”) e inferior (“ $g \geq f$ ”), respectivamente. Por sus propias definiciones, las constantes multiplicativas y los términos de menor orden quedan absorbidos en constantes c_{inf} y c_{sup} , por lo que no alteran la clase asintótica a la que pertenece una función. Esto justifica **representar por convenio cada conjunto mediante su representante canónico más sencillo: la función dominante más simple que ignora constantes multiplicativas y términos de menor orden**. Por ej. es correcto escribir $\mathcal{O}(6 \cdot n + 2)$ o $\mathcal{O}(n^2 + 20 \cdot n + 1500)$, pero por convenio se escribe $\mathcal{O}(n)$ y $\mathcal{O}(n^2)$, respectivamente; análogo para $\Omega(\cdot)$ y $\Theta(\cdot)$. Aunque esto antes se ilustró con el ejemplo de $T_A(n) = n^2 + 20 \cdot n + 1500$, con las definiciones anteriores puede justificarse formalmente. Por ejemplo, se puede demostrar que cualquier función lineal $g : \mathbb{N}_0 \rightarrow \mathbb{R}_{\geq 0}$, $g(n) = c_1 \cdot n + c_0$ cumple $g(n) \in \mathcal{O}(n)$: por definición de $\mathcal{O}(\cdot)$, hay que encontrar constantes $c_{sup} \in \mathbb{R}^+$ y $n_0 \in \mathbb{N}^+$ tales que: $\forall n \geq n_0 : 0 \leq c_1 \cdot n + c_0 \leq c_{sup} \cdot n$:

Por un lado, la hipótesis $g : \mathbb{N}_0 \rightarrow \mathbb{R}_{\geq 0}$ asegura que $\forall n \in \mathbb{N}_0 : g(n) \geq 0$, por tanto la desigualdad $0 \leq c_1 \cdot n + c_0 = g(n)$ se cumple con independencia de los valores de c_{sup} y n_0 .

Por otro lado, si $n \geq 1$ entonces $g(n) = c_1 \cdot n + c_0 \leq |c_1| \cdot n + |c_0| \leq |c_1| \cdot n + |c_0| \cdot n = (|c_1| + |c_0|) \cdot n$, siendo $|\cdot|$ el valor absoluto. Por tanto, basta tomar $c_{sup} = |c_1| + |c_0|$ y $n_0 = 1$ para tener $\forall n \geq n_0 : c_1 \cdot n + c_0 \leq c_{sup} \cdot n$.

En general, para cualquier función polinómica $g(n) = c_k \cdot n^k + c_{k-1} \cdot n^{k-1} + \dots + c_1 \cdot n + c_0 = \sum_{i=0}^k c_i \cdot n^i$ con $g : \mathbb{N}_0 \rightarrow \mathbb{R}_{\geq 0}$, puede probarse análogamente que $g(n) \in \mathcal{O}(n^k)$ tomando los valores $n_0 = 1$ y c_{sup} como la suma de los valores absolutos de los coeficientes, $c_{sup} = |c_k| + |c_{k-1}| + \dots + |c_1| + |c_0| = \sum_{i=0}^k |c_i|$. Nótese que para demostrarlo para $\Theta(\cdot)$, habría que realizar una demostración análoga para $\Omega(\cdot)$.

2.3.2. Álgebra de la notación de Landau

En este apartado se presentan algunas de las operaciones y propiedades más relevantes de las notaciones $\mathcal{O}(\cdot)$, $\Omega(\cdot)$ y $\Theta(\cdot)$. Posteriormente, se verá que estas operaciones y propiedades son de utilidad para calcular la complejidad computacional de algoritmos.

La definición 7 introduce la **suma y el producto de órdenes de complejidad**:

Definición 7 (Suma y producto de órdenes de complejidad). Sean $f_1, f_2 : \mathbb{N}_0 \rightarrow \mathbb{R}_{\geq 0}$. Se definen y denotan la **suma y el producto de órdenes de complejidad** de la siguiente manera:

$$\mathcal{O}(f_1(n)) + \mathcal{O}(f_2(n)) = \{g_1(n) + g_2(n) \mid g_1 \in \mathcal{O}(f_1(n)) \wedge g_2 \in \mathcal{O}(f_2(n))\}$$

$$\mathcal{O}(f_1(n)) \cdot \mathcal{O}(f_2(n)) = \{g_1(n) \cdot g_2(n) \mid g_1 \in \mathcal{O}(f_1(n)) \wedge g_2 \in \mathcal{O}(f_2(n))\}$$

Las anteriores definiciones pueden aplicarse análogamente para $\Omega(\cdot)$ y $\Theta(\cdot)$.

La anterior definición establece la suma/producto entre dos órdenes de complejidad $\mathcal{O}(f_1(n))$ y $\mathcal{O}(f_2(n))$ como el conjunto de todas las funciones que resultan de sumar/multiplicar una función de $\mathcal{O}(f_1(n))$ con otra función de $\mathcal{O}(f_2(n))$. La suma y producto de funciones se define de la siguiente manera: dadas dos funciones $f, g : \mathbb{N}_0 \rightarrow \mathbb{R}_{\geq 0}$, la función suma se define como $(f + g)(n) = f(n) + g(n)$ y la función producto se define como $(f \cdot g)(n) = f(n) \cdot g(n)$.

En este contexto es posible encontrarse con expresiones como, por ejemplo, $f(n) = 2 \cdot n^3 + \mathcal{O}(n^2)$. Teniendo en cuenta la definición anterior, esto significa que existe una función $g(n) \in \mathcal{O}(n^2)$ tal que $f(n) = 2 \cdot n^3 + g(n)$. Como $2 \cdot n^3 \in \mathcal{O}(n^3)$, también sería válido escribir $f(n) = \mathcal{O}(n^3) + \mathcal{O}(n^2)$. Y dado que n^3 es dominante con respecto a n^2 , sería válido escribir $f(n) = \mathcal{O}(n^3)$. Por este motivo, en el contexto de la complejidad computacional suele utilizarse las notaciones $g = \mathcal{O}(f)$, $g = \Omega(f)$ y $g = \Theta(f)$ de manera equivalente a $g \in \mathcal{O}(f)$, $g \in \Omega(f)$ y $g \in \Theta(f)$, respectivamente.

La proposición 1 presenta algunas de las propiedades de la notación de Landau más importantes:

Proposición 1 (Propiedades de $\mathcal{O}(\cdot)$, $\Omega(\cdot)$ y $\Theta(\cdot)$). Para cualesquiera funciones $f, g, h : \mathbb{N}_0 \rightarrow \mathbb{R}_{\geq 0}$ se cumplen las siguientes propiedades:

- (i) **Reflexividad:** $f \in \mathcal{O}(f)$, $f \in \Omega(f)$, $f \in \Theta(f)$
- (ii) **Dualidad entre $\mathcal{O}(\cdot)$ y $\Omega(\cdot)$:** $g \in \mathcal{O}(f) \iff f \in \Omega(g)$
- (iii) **Simetría de $\Theta(\cdot)$:** $g \in \Theta(f) \iff f \in \Theta(g)$
- (iv) **Transitividad:** $f \in \mathcal{O}(g) \wedge g \in \mathcal{O}(h) \implies f \in \mathcal{O}(h)$.
- (v) **Producto por escalares:** $\forall c \in \mathbb{R}^+ : \mathcal{O}(c \cdot f) = \mathcal{O}(f)$.
- (vi) **Regla de la suma:** $\mathcal{O}(f) + \mathcal{O}(g) = \mathcal{O}(f + g) = \mathcal{O}(\max\{f, g\})$.
- (vii) **Regla del producto:** $\mathcal{O}(f) \cdot \mathcal{O}(g) = \mathcal{O}(f \cdot g)$.

Las propiedades (iv)-(vii) se cumplen análogamente para $\Omega(\cdot)$ y $\Theta(\cdot)$.

La demostración formal de la proposición puede llevarse a cabo manejando las definiciones de $\mathcal{O}(\cdot)$, $\Omega(\cdot)$ y $\Theta(\cdot)$ de forma similar al ejemplo anterior a este apartado y se dejan como ejercicio. Las propiedades (i)-(iv) formalizan que $\Theta(\cdot)$ induce una relación de equivalencia (reflexiva, simétrica y transitiva), mientras que $\mathcal{O}(\cdot)$ y $\Omega(\cdot)$ no son simétricas, aunque están dualmente relacionadas. La propiedad (v) formaliza que el papel de las constantes multiplicativas puede ignorarse en la notación de Landau. La propiedad (vi) establece que el orden de complejidad de una suma de funciones viene dado por la función dominante, es decir, por el máximo. Por último, la propiedad (vii) establece que el orden de complejidad de un producto de funciones es el producto de sus órdenes de complejidad, es decir, viene dado por el producto de las funciones dominantes.

2.3.3. Jerarquía de órdenes de complejidad

Con las herramientas presentadas con anterioridad es posible acotar el coste temporal de un algoritmo $\mathcal{A}$ de la siguiente manera: (i) encontrando una función f_{inf} tal que $T_{\mathcal{A}}^{mejor} \in \Omega(f_{inf})$, esto es una cota inferior del coste temporal del algoritmo en el caso mejor: $\mathcal{A}$ no puede ser más rápido que $\Omega(f_{inf})$; y (ii) encontrando una función f_{sup} tal que $T_{\mathcal{A}}^{peor} \in \mathcal{O}(f_{sup})$, esto es una cota superior del coste temporal del algoritmo en el caso peor: $\mathcal{A}$ no puede ser más lento que $\mathcal{O}(f_{sup})$. Por ejemplo, en la búsqueda lineal (Algoritmo 15) vimos que $T_{\mathcal{A}}^{mejor} \in \Omega(1)$ (en realidad $\Theta(1)$) mientras que $T_{\mathcal{A}}^{peor} \in \mathcal{O}(n)$ (en realidad $\Theta(n)$), siendo n el número de componentes del array dado como parámetro.

Salvo que se quiera realizar un estudio del coste especialmente fino (por ej. caso promedio), en la práctica lo más habitual es calcular la **complejidad computacional en el caso peor** empleando la notación $\mathcal{O}(\cdot)$, ya que nos proporciona una estimación segura del coste temporal máximo que un algoritmo puede tardar, es más sencilla de calcular que otros análisis de complejidad más detallados y es mucho más informativo que el coste mínimo en el mejor de los casos. Esto permite **clasificar algoritmos de acuerdo con su coste asintótico temporal en el caso peor**. Dado que las funciones con un crecimiento asintótico menor siempre tendrán como cota superior a otras funciones con un crecimiento asintótico mayor, esta clasificación suele presentarse mediante una cadena de subconjuntos (estrictos) empleando, para cada tipo de función, su representante canónico más sencillo. Esta cadena se denomina **jerarquía de órdenes de complejidad** y es la siguiente:

$$\begin{aligned} \mathcal{O}(1) &\subset \mathcal{O}(\log^{(k)} n) \subset \dots \subset \mathcal{O}(\log n) \subset \mathcal{O}((\log n)^k) \subset \dots \subset \mathcal{O}(n^\varepsilon) \subset \dots \subset \mathcal{O}(n) \subset \\ &\subset \mathcal{O}(n \log n) \subset \dots \subset \mathcal{O}(n^2) \subset \mathcal{O}(n^2 \log n) \subset \dots \subset \mathcal{O}(n^k) \subset \mathcal{O}(n^k \log n) \subset \dots \\ &\dots \subset \mathcal{O}(2^n) \subset \mathcal{O}(3^n) \subset \dots \subset \mathcal{O}(c^n) \subset \dots \subset \mathcal{O}(n!) \subset \mathcal{O}(n^n) \end{aligned}$$

donde: $k \in \mathbb{N}^+$ y $c \in \mathbb{R}^+$ con $c > 1$ y $\varepsilon \in (0, 1) \subset \mathbb{R}^+$, por ej. $\varepsilon = 1/2, 1/3, \dots$. Por otro lado, $\log^{(k)} n$ representa el *logaritmo iterado*, i.e. aplicar k veces la función logaritmo a un número n , que se define recursivamente de la siguiente manera:

$$\text{Caso base: } \log^{(1)} n = \log n,$$

$$\text{Caso recursivo: } \log^{(k)} n = \log(\log^{(k-1)} n).$$

Dado que la jerarquía de órdenes de complejidad es una cadena de inclusiones, se puede afirmar que si la función de coste de un algoritmo se encuentra en $\mathcal{O}(n)$, entonces también se encuentra en $\mathcal{O}(n^2)$, o en $\mathcal{O}(2^n)$, etc. Sin embargo, referirse a cotas aún mayores no se considera correcto dado que es impreciso y poco útil. Por este motivo, cuando se pregunta por el coste computacional de un algoritmo $\mathcal{A}$ en el caso peor, el convenio es responder que es $\mathcal{O}(f(n))$ siendo $f(n)$ **la cota superior de menor orden de complejidad que seamos capaces de calcular**.

Los órdenes de complejidad de esta jerarquía permiten clasificar un algoritmo $\mathcal{A}$ según sea su coste computacional en el caso peor, i.e. $T_{\mathcal{A}}^{peor} \in \mathcal{O}(f)$. Se usa la siguiente terminología, destacando en negrita aquellos más frecuentes:

- $\mathcal{O}(1)$: **coste constante/algoritmos constantes**. Son aquellos que no dependen del tamaño de los datos de entrada e involucran solamente operaciones elementales.
- $\mathcal{O}(\log^{(k)} n)$, $k > 1$: **coste sublogarítmico/algoritmos sublogarítmicos**. Presentan un crecimiento del coste extremadamente lento, cercano al constante cuanto mayor sea k .
- $\mathcal{O}(\log n)$: **coste logarítmico/algoritmos logarítmicos**. Considerados algoritmos muy eficientes. Nótese que la base del logaritmo se ignora por ser independiente del coste. Esto se debe a que la fórmula de cambio de base entre logaritmos es la siguiente: si $a, b \in \mathbb{R}^+$, con $a, b \neq 1$, entonces $\log_a n = \frac{\log_b n}{\log_b a}$ y $\frac{1}{\log_b a}$ es una constante que, por la propiedad (v) de la Proposición 1, puede ignorarse teniendo entonces $\mathcal{O}(\log_a n) = \mathcal{O}(\log_b n)$.
- $\mathcal{O}((\log n)^k)$, $k > 1$: **coste polilogarítmico/algoritmos polilogarítmicos**.
- $\mathcal{O}(n^\varepsilon)$, $\varepsilon \in (0, 1) \subset \mathbb{R}^+$: **coste sublineal/algoritmos sublineales**. Su coste crece más rápido que el logarítmico pero más lento que el lineal. En particular, esta clase incluye costes de raíces k -ésimas de n : $\mathcal{O}(n^{1/2}) = \mathcal{O}(\sqrt{n})$, $\mathcal{O}(n^{1/3}) = \mathcal{O}(\sqrt[3]{n})$, etc.
- $\mathcal{O}(n^k)$, $k \in \mathbb{N}^+$: **coste polinómico/algoritmos polinómicos (o polinomiales)** de grado k . Son considerados eficientes en muchos contextos prácticos. En particular, $\mathcal{O}(n)$ se denomina **coste lineal (algoritmos lineales)**, $\mathcal{O}(n^2)$ se denomina **coste cuadrático (algoritmos cuadráticos)** y $\mathcal{O}(n^3)$ se denomina **coste cúbico (algoritmos cúbicos)**.
- $\mathcal{O}(n^k \log n)$, $k \in \mathbb{N}^+$: **costes/algoritmos polinómicos con factor logarítmico**. En particular, $\mathcal{O}(n \log n)$ se denomina **cuasi-lineal** y es el caso más común. A medida que crece el tamaño de n , estos costes se asemejan cada vez más a los costes polinómicos porque la componente $\log n$ tiene menor peso.
- $\mathcal{O}(c^n)$, $c \in \mathbb{R}^+$, $c > 1$: **coste exponencial/algoritmos exponenciales** de base c . Su crecimiento es muy rápido, lo que los hace ineficientes para tamaños grandes de entrada.
- $\mathcal{O}(n!)$: **coste factorial/algoritmos factoriales**. Aparecen comúnmente en problemas que involucran cálculo de permutaciones y combinatoria exhaustiva.
- $\mathcal{O}(n^n)$: **coste superexponencial/algoritmos superexponenciales** (también se usa la denominación hiperexponencial). Representan crecimientos extremadamente rápidos.

En general, se suele denominar *algoritmos tratables* a aquellos cuyo coste computacional en el caso peor es a lo sumo polinomial. En determinados contextos, también se incluyen los algoritmos polinomiales con factor logarítmico, dado que su crecimiento asintótico resulta cercano al polinómico para tamaños de entrada grandes. Por otro lado, se suele denominar *algoritmos intratables* a aquellos con un coste computacional en el caso peor exponencial o superior, lo que implica que su ejecución resulta inviable en la práctica incluso para instancias de tamaño moderado. No obstante, esta consideración es dependiente del contexto. En aplicaciones donde la eficiencia es un requisito crítico, puede adoptarse una noción más restrictiva de *tratabilidad*, limitada a algoritmos con complejidad lineal o incluso inferior.

La jerarquía de órdenes de complejidad engloba propiedades interesantes como las siguientes:

$$\forall k \in \mathbb{N}^+ : \mathcal{O}(\log^{(k+1)} n) \subset \mathcal{O}(\log^{(k)} n), \text{ i.e. mayor iteración de log. } \implies \text{menor crecimiento}$$

$\forall k \in \mathbb{N}^+ : \mathcal{O}((\log n)^k) \subset \mathcal{O}((\log n)^{k+1})$, i.e. mayor potencia de log. $\implies$ mayor crecimiento

$\forall k \in \mathbb{N}^+ \forall \varepsilon \in (0, 1) : \mathcal{O}((\log n)^k) \subset \mathcal{O}(n^\varepsilon)$, i.e. potencias log. crecen menos que potencias de n

$\forall a, b, 0 < a < b : \mathcal{O}(n^a) \subset \mathcal{O}(n^b)$, i.e. mayor exponente polinomial $\implies$ mayor crecimiento

$\forall k \in \mathbb{N}^+ : \mathcal{O}(n^k) \subset \mathcal{O}(n^k \log n)$, i.e. multiplicar por log. $\implies$ mayor crecimiento (ligero)

$\forall a, b, 1 < a < b : \mathcal{O}(a^n) \subset \mathcal{O}(b^n)$, i.e. mayor base exponencial $\implies$ mayor crecimiento

Por último, cabe señalar que la jerarquía de órdenes de complejidad se aplica totalmente para la notación $\Theta(\cdot)$. Sin embargo, en el caso de la notación $\Omega(\cdot)$, la situación es inversa a $\mathcal{O}(\cdot)$, i.e., se cumple la jerarquía opuesta: $\Omega(1) \supset \dots \supset \Omega(n^n)$. Por analogía, el convenio es escribir $\Omega(f(n))$ cuando $f(n)$ es la cota inferior de mayor orden de complejidad que seamos capaces de calcular.

2.4. Cálculo del coste computacional de algoritmos

El objetivo de esta sección es aplicar los conceptos teóricos presentados anteriormente para calcular el coste computacional de algoritmos. En esta asignatura nos centraremos en el **análisis del coste computacional en el caso peor**, lo cual implica: (i) identificar qué entradas del algoritmo provocan la ejecución de un mayor número de operaciones elementales; y (ii) aplicar reglas y técnicas que permitan determinar la clase más precisa posible a la que pertenece el algoritmo dentro de la jerarquía de órdenes de complejidad⁹.

Un algoritmo $\mathcal{A}$ puede verse como una secuencia finita de sentencias $\mathcal{A} \equiv S_1; S_2; \dots; S_m$ donde cada sentencia S_i puede consistir en una operación elemental, un condicional, un bucle, una llamada a un algoritmo, etc. Si alguna de estas sentencias S_i consiste en una llamada al propio algoritmo $\mathcal{A}$, se dice que $\mathcal{A}$ es un **algoritmo recursivo**¹⁰, mientras que, en caso contrario, se dice que $\mathcal{A}$ es un **algoritmo iterativo**. Esta sección se divide en dos apartados principales que abordan el cálculo del coste computacional para cada tipo de algoritmo:

- El primer apartado se centra en el cálculo del **coste computacional de algoritmos iterativos**. En particular, se presentan reglas prácticas para analizar el coste de distintas sentencias S_i : estructuras condicionales (**if-then-else**, **case**), bucles (**for**, **while**), etc.
- El segundo apartado se centra en el cálculo del **coste computacional de algoritmos recursivos**. Las funciones de coste de estos algoritmos se denominan recurrencias y también se definen recursivamente. En esta asignatura estudiaremos dos tipos habituales de recurrencias especialmente relevantes para los que existen resultados matemáticos que permiten determinar su coste computacional sin necesidad de realizar cálculos complejos.

⁹No obstante, en esta asignatura nos conformaremos con cotas menos ajustadas para el caso de ciertos tipos de algoritmos recursivos.

¹⁰Incluyendo el caso de la *recursión mutua*, i.e.: $\mathcal{A}$ llama a otro algoritmo $\mathcal{A}'$ que, a su vez, llama a $\mathcal{A}$.

2.4.1. Coste computacional de algoritmos iterativos

Denotaremos por $T_S(n)$ a la función de coste de una sentencia S asumiendo el caso peor. A continuación, se indican **reglas de cálculo de costes para cada tipo de sentencia**:

- **Operaciones elementales.** Si $S \equiv \text{op. elemental}$, entonces $T_S(n) \in \mathcal{O}(1)$. Se recuerda que las ops. elementales consideradas en esta asignatura se listan en el apartado 2.1.2.
- **Sentencias condicionales.** Si $S \equiv \text{if } C \text{ then } S_1 \text{ else } S_2$, se tiene que la condición C siempre se evalúa y, a continuación, se ejecuta la rama **if** si C se evalúa verdadera, o bien la rama **else** si C se evalúa falsa. Por tanto, la función $T_S(n)$ es:

$$T_S(n) = \begin{cases} T_C(n) + T_{S_1}(n) & \text{si } C = \text{verdadero} \\ T_C(n) + T_{S_2}(n) & \text{si } C = \text{falso} \end{cases}$$

Como se asume el caso peor, la fórmula anterior puede simplificarse tomando la función de coste dominante de ambas ramas, i.e., $T_S(n) = T_C(n) + \max\{T_{S_1}(n), T_{S_2}(n)\}$. Por tanto $T_S(n) \in \mathcal{O}(T_C(n)) + \mathcal{O}(\max\{T_{S_1}(n), T_{S_2}(n)\}) = \mathcal{O}(\max\{T_C(n), T_{S_1}(n), T_{S_2}(n)\})$, donde la última igualdad viene de aplicar la regla de la suma de la Proposición 1.

El caso de las sentencias **if-then-else** anidadas se tiene si S_1 o S_2 también son sentencias **if-then-else** (y sucesivamente), de manera que la regla anterior se aplica recursivamente teniendo que el coste asintótico temporal viene determinado por el orden de complejidad de la función de coste dominante de todas las condiciones y ramas involucradas.

También existe otro tipo de sentencias condicionales: las sentencias **switch-case**, denominadas **match-case** en Python:

$$S \equiv \text{switch}(E) \{ \\ \quad \text{case } v_1 : \quad S_1 \\ \quad \text{case } v_2 : \quad S_2 \\ \quad \quad \quad \vdots \\ \quad \text{case } v_k : \quad S_k \\ \quad \text{default} : \quad S_d \\ \}$$

Por simplicidad, asumiremos que este tipo de sentencias tratan solamente con tipos básicos y enumerados, lo cual es cierto en varios lenguajes de programación, aunque no es así en Python. El funcionamiento de este tipo de sentencias es el siguiente:

- Evaluación de la expresión E : E es una expresión que se evalúa a un cierto valor v de tipo básico o enumerado. Por ej. E puede ser una variable, en cuyo caso su evaluación (un acceso a memoria) tiene coste $T_E \in \mathcal{O}(1)$, pero también puede ser una expresión más compleja, por ej. $E \equiv A[i]$, equivalentes en otras estructuras de datos, una llamada a una función que devuelve un valor de tipo básico, etc. En esos casos T_E no tiene por qué ser de coste constante.

- Selección de casos: Se compara el valor v con cada valor constante v_i , de manera que si $v = v_i$ entonces se ejecuta su sentencia asociada S_i . Al asumir que v y los valores v_i son de tipo básico o enumerado, cada comparación $v = v_i$ tiene coste $\mathcal{O}(1)$. En el peor caso se realizan k comparaciones, pero como el número de casos es fijo podemos asumir que k es independiente del tamaño de la entrada, de manera que el coste total de las comparaciones es $\mathcal{O}(k) = \mathcal{O}(1)$.
- Caso por defecto: En caso de que el valor v no coincida con ningún valor v_i entonces se ejecuta la sentencia S_d asociada al caso por defecto **default**, que en Python se escribe como `"case _"`.

Bajo las asunciones anteriores y por razonamientos similares a los de las sentencias **if-then-else**, se tiene que $T_S(n) \in \mathcal{O}(\max\{T_E(n), T_{S_1}(n), T_{S_2}(n), \dots, T_{S_k}(n), T_{S_d}(n)\})$.

- **Bucles**. Anteriormente vimos en el Algoritmo 14 la equivalencia entre bucles **for** y **while** donde se muestra que ambos tipos de bucles tienen en común los siguientes elementos:
 - **Inicialización (ini)**: se definen los valores iniciales de las variables de control del bucle.
 - **Condición (C)**: expresión booleana dependiente de alguna variable de control. Define el criterio de parada del bucle cuando se evalúa falsa.
 - **Cuerpo del bucle (S')**: conjunto de instrucciones ejecutadas en cada iteración del bucle.
 - **Actualización (act)**: modificación de las variables de control de manera que la ejecución se acerque a un estado en el que deja de cumplirse la condición.

En general, la complejidad de un bucle se obtiene multiplicando el número de iteraciones $I(n)$ por el coste de cada iteración.

El coste de cada iteración viene determinado por:

- T_{ini} función de coste de la inicialización (solo en bucles **for**),
- T_C función de coste de evaluar la condición,
- $T_{S'}$ función de coste de ejecutar el cuerpo del bucle,
- T_{act} función de coste de ejecutar la actualización.

Si $S \equiv \text{while } C \text{ do } S'^{11}$, entonces $T_S(n) = (I(n) + 1) \cdot T_C(n) + I(n) \cdot (T_{S'}(n) + T_{act}(n))$, puesto que la condición se chequea una vez adicional cuando se evalúa falsa. Se aplica la siguiente regla:

$$T_S(n) \in \mathcal{O}\left(I(n) \cdot \max\{T_C(n), T_{S'}(n), T_{act}(n)\}\right)$$

Nótese que en los bucles **while** la inicialización se define externamente, por lo que su coste se calcula por separado.

Si $S \equiv \text{for}(\text{ini}; C; \text{act})\{S'\}$, la inicialización se incluye dentro del bucle y se ejecuta una única vez, por lo que su coste se suma al producto del número de iteraciones por el coste

¹¹Para $S \equiv \text{do } S' \text{ while } C$ también valdría. Este tipo de bucle **while** asegura al menos una ejecución de la sentencia S' , mientras que el bucle **while** habitual siempre evalúa la condición en primer lugar.

de cada iteración, es decir: $T_S(n) = T_{ini}(n) + (I(n) + 1) \cdot T_C(n) + I(n) \cdot (T_{S'}(n) + T_{act}(n))$. Se aplica la siguiente regla:

$$T_S(n) \in \mathcal{O}(\max\{T_{ini}(n), I(n) \cdot \max\{T_C(n), T_{S'}(n), T_{act}(n)\}\})$$

En muchos casos prácticos, la inicialización, la condición y la actualización tienen coste constante, es decir, $T_{ini}, T_C(n), T_{act}(n) \in \mathcal{O}(1)$, por lo que la expresión suele simplificarse a $T_S(n) \in \mathcal{O}(I(n) \cdot T_{S'}(n))$ para ambos tipos de bucles. Sin embargo, siempre hay que comprobar el coste de todas ellas. Por ej. una condición podría ser una llamada a la función booleana `busquedaLineal` (Algoritmo 15), cuyo coste en el caso peor no es constante.

Por otro lado, el número de iteraciones $I(n)$ viene determinado por tres elementos dependientes de las variables de control $\bar{x}$, ilustrados con el ejemplo habitual de bucles mostrado en el Algoritmo 14:

- la inicialización de la/s variable/s de control: $\bar{x} = \bar{x}_0$. Ejemplo habitual: $i = 1$.
- la condición de parada, dependiente de alguna variable de control, i.e. $C(\bar{x})$. Ejemplo habitual: $C \equiv i \leq n$.
- la regla de actualización: $\bar{x}_{t+1} = h(\bar{x}_t)$, donde $\bar{x}_t$ es el valor de la variable de control en la iteración t y h es una operación que modifica la/s variable/s de control de una iteración t a la siguiente iteración $t + 1$. Ejemplo habitual: $i = i + 1$.

Por tanto: $I(n) = \min\{t \in \mathbb{N}_0 \mid C(\bar{x}_t) = \text{falso}\}$. En el ejemplo habitual, $I(n) = n$. En particular, cuando la regla de actualización consiste en un incremento/decremento de la variable de control, el número de iteraciones suele tener coste $I(n) \in \mathcal{O}(n)$ y si la regla de actualización consiste en una multiplicación/división de la variable de control, el número de iteraciones suele tener coste $I(n) \in \mathcal{O}(\log n)$. No obstante, depende de cada caso y el cálculo puede llegar a ser muy complejo.

Los bucles son, como los bloques `if-then-else`, otro tipo de sentencias que se definen recursivamente puesto que S' también es una sentencia. En particular, el caso de los bucles anidados se tiene cuando S' es otro bucle y así sucesivamente. El Algoritmo 16 muestra esta situación con k bucles `for` anidados:

Algoritmo 16 k bucles `for` anidados

```

Para  $i_1 = 1$  hasta  $n_1$  hacer
  Para  $i_2 = 1$  hasta  $n_2$  hacer
     $\vdots$ 
    Para  $i_k = 1$  hasta  $n_k$  hacer
       $S'$ 

  Fin Para
   $\vdots$ 
Fin Para
Fin Para

```

En este caso, la regla anterior se aplica recursivamente. Como las inicializaciones, condiciones y actualizaciones tienen coste constante (asignaciones, comparaciones e incrementos

respectivamente), el coste recae en el número de iteraciones de todos los bucles junto al coste de la sentencia S' . Denotando por $I_j(n)$ al número de iteraciones del j -ésimo bucle, se tiene:

$$T_S(n) = (I_1(n) \cdot I_2(n) \cdot \dots \cdot I_k(n)) \cdot T_{S'}(n) = T_{S'}(n) \cdot \prod_{j=1}^k I_j(n)$$

Si $n_1 = n_2 = \dots = n_k = n$, el coste viene dado por $\mathcal{O}(n^k) \cdot \mathcal{O}(T_{S'}) = \mathcal{O}(n^k \cdot T_{S'})$, aplicando la regla del producto de la Proposición 1. Nótese que **en este ejemplo concreto el número de iteraciones de cada bucle es independiente del número de iteraciones del resto de bucles**. Sin embargo, también es frecuente encontrarse con bucles anidados en los que esto no es así. El Algoritmo 17 muestra un ejemplo sencillo de esta situación:

Algoritmo 17 Bucles **for** con iteraciones dependientes

```

Para  $i = 1$  hasta  $n$  hacer
  Para  $j = 1$  hasta  $i$  hacer
     $S'$ 
  Fin Para
Fin Para

```

Como se puede ver, el segundo bucle **for** (bucle interno) tiene un número de iteraciones dependiente del primero (bucle externo), ya que el valor final de j depende de i . En estos casos, el número total de ejecuciones de la sentencia S' no se obtiene mediante un producto de iteraciones, sino sumando, para cada iteración del bucle externo, cuántas veces se ejecuta el bucle interno, es decir:

$$T_S(n) = \sum_{i=1}^{I_1(n)} I_2(i) \cdot T_{S'}(n)$$

donde $I_1(n) = n$. En el Algoritmo 17, para cada valor de i , el índice j toma los valores $1, 2, \dots, i$, por lo que el bucle interno realiza $I_2(i) = i$ iteraciones. Por tanto:

$$T_S(n) = \sum_{i=1}^n i \cdot T_{S'}(n) = \frac{n \cdot (n+1)}{2} \cdot T_{S'}(n)$$

donde la igualdad $\sum_{i=1}^n i = \frac{n \cdot (n+1)}{2}$ puede demostrarse por inducción sobre $n \in \mathbb{N}^+$. Por tanto: $T_S(n) \in \mathcal{O}(n^2 \cdot T_{S'})$. Nótese que se obtendría el mismo resultado si el índice j del bucle interno recorriese desde i hasta n . Por otro lado, en este ejemplo las reglas de actualización de ambos índices consisten en incrementos, pero si la actualización fuese diferente podría dar lugar a distintos costes.

- **Llamadas a algoritmos.** $S \equiv \mathcal{A}'(E_1, E_2, \dots, E_k)$ es el caso de llamadas a funciones o procedimientos, donde $\mathcal{A}'$ es un algoritmo distinto al llamante en el escenario de los algoritmos iterativos. Al igual que en el caso de las sentencias **switch-case**, hay una serie de expresiones E_i (los parámetros del algoritmo) con la diferencia de que no es necesario que se evalúen a un valor de tipo básico.

Al realizar una llamada a un algoritmo, primero se evalúan sus parámetros y, a continuación, se ejecuta el algoritmo. Por tanto, el coste se obtiene sumando los costes asociados: $T_S(n) = (T_{E_1}(n) + T_{E_2}(n) + \cdots + T_{E_k}(n)) + T_{A'}(n)$, de manera que aplicando la regla de la suma se tiene:

$$T_S(n) \in \mathcal{O}(\text{máx}\{T_{E_1}, T_{E_2}, \dots, T_{E_k}, T_{A'}\})$$

- **Secuencia de sentencias.** $S \equiv S_1; S_2$ significa que primero se ejecuta la secuencia S_1 y posteriormente la secuencia S_2 , de manera que $T_S(n) = T_{S_1}(n) + T_{S_2}(n)$. Por tanto: $T_S \in \mathcal{O}(\text{máx}\{T_{S_1}, T_{S_2}\})$.

Una vez vistas las reglas anteriores, como un algoritmo iterativo puede verse como una secuencia de sentencias $\mathcal{A} \equiv S_1; S_2; \dots; S_m$ sin llamadas recursivas, el cálculo de su coste se obtiene como aplicación recursiva de la última regla: $T_{\mathcal{A}}(n) = T_{S_1}(n) + T_{S_2}(n) + \cdots + T_{S_m}(n)$. Por tanto:

$$T_{\mathcal{A}} \in \mathcal{O}(\text{máx}\{T_{S_1}, T_{S_2}, \dots, T_{S_m}\})$$

2.4.2. Coste computacional de algoritmos recursivos

Un algoritmo $\mathcal{A} \equiv S_1; S_2; \dots; S_m$ es recursivo si alguna de sus sentencias S_i implica o consiste en una llamada a sí mismo (*llamada recursiva*). Las soluciones recursivas (como vimos en el Capítulo 1) constan de al menos un caso base, donde no aparecen llamadas recursivas, y al menos un caso recursivo, donde aparece alguna llamada recursiva. Una solución recursiva termina cuando las sucesivas llamadas recursivas convergen en algún caso base, que actúa como criterio de parada de la recursión.

La función de coste de un algoritmo recursivo se define de forma recursiva, reflejando la estructura del propio algoritmo. Este tipo de funciones se denominan **ecuaciones de recurrencia** o, simplemente, **recurrencias** y tienen la siguiente forma:

$$T_{\mathcal{A}}(n) = \begin{cases} T_{cb}(n) & \text{si } n \leq n_0 \\ \sum_{i=1}^k a_i T_{\mathcal{A}}(b_i(n)) + T_{nr}(n) & \text{si } n > n_0 \end{cases}$$

donde:

- $T_{cb}(n)$ es la función de coste del caso base. Su coste se obtiene aplicando las reglas vistas en el apartado anterior para algoritmos iterativos, aunque suele ser $T_{cb}(n) \in \Theta(1)$.
- a_i es el número de llamadas recursivas a subproblemas de tamaño $b_i(n)$. Supondremos que los valores a_i son constantes, aunque se podría dar el caso de que dependiesen de n , posibilidad que no abordaremos en esta asignatura. El número total de llamadas recursivas es la suma de los valores a_i : $a = \sum_{i=1}^k a_i$.
- $b_i(n)$ es cada tamaño de subproblema diferente para los que se aplica recursión. En esta asignatura abordaremos dos casos: (i) sustracción: $b_i(n) = n - b_i$, con $b_i \in \mathbb{N}^+$; (ii) división: $b_i(n) = \frac{n}{b_i}$, con $b_i \in \mathbb{N}^+$ tales que $b_i > 1$.

- $T_{nr}(n)$ es la función de coste de todas las operaciones no recursivas que se realizan en el caso recursivo. Al igual que con $T_{cb}(n)$, su coste se obtiene aplicando las reglas vistas para algoritmos iterativos.

Por ejemplo, en el caso del Algoritmo 3 del capítulo anterior que calcula el n -ésimo número de la sucesión de Fibonacci, su ecuación de recurrencia es la siguiente:

$$T_{fib}(n) = \begin{cases} \Theta(1) & \text{si } n \leq 1 \\ T_{fib}(n-1) + T_{fib}(n-2) + \Theta(1) & \text{si } n > 1 \end{cases}$$

El caso base consiste en una sentencia de retorno de resultado, por lo que tiene coste $\Theta(1)$. En el caso recursivo hay $k = 2$ tamaños de subproblemas diferentes: $b_1(n) = n - 1$ con $a_1 = 1$ llamada recursiva y $b_2(n) = n - 2$ con $a_2 = 1$ llamada recursiva, de modo que en total hay $a = a_1 + a_2 = 2$ llamadas recursivas. Las operaciones no recursivas en el caso recursivo son las restas $n - 1$ y $n - 2$ y la suma de los resultados de las dos llamadas recursivas, por tanto tiene coste $\Theta(1)$.

Para las recurrencias que abordaremos en esta asignatura asumiremos dos condiciones adicionales:

- Las funciones de coste son no decrecientes, i.e.: $\forall n, m \in \mathbb{N}_0 : n \leq m \implies T(n) \leq T(m)$.
- Los tamaños de los subproblemas $b_i(n)$ siempre son menores que n , por ej. $n - 1$, o $\frac{n}{2}$.

Bajo estas asunciones se tiene lo siguiente:

$$\sum_{i=1}^k a_i T_{\mathcal{A}}(b_i(n)) \leq a \cdot T_{\mathcal{A}}(b^*(n))$$

siendo $a = \sum_{i=1}^k a_i$ el número total de llamadas recursivas y $b^*(n) = \max\{b_i(n) \in \mathbb{N}^+ \mid i = 1, 2, \dots, k\}$ el tamaño de subproblema más grande. Esto permite acotar todos los subproblemas por el peor caso (el subproblema más grande) y además permite simplificar la forma de las recurrencias vista anteriormente a una expresión más sencilla que asumiremos en lo sucesivo:

$$T_{\mathcal{A}}(n) = \begin{cases} T_{cb}(n) & \text{si } n \leq n_0 \\ a \cdot T_{\mathcal{A}}(b^*(n)) + T_{nr}(n) & \text{si } n > n_0 \end{cases}$$

Esta simplificación facilita el análisis aunque tiene la desventaja de que proporciona cotas superiores menos ajustadas.

Como se comentó anteriormente, en esta asignatura solamente nos centraremos en dos tipos de recurrencias bastante frecuentes caracterizadas por su factor $b_i(n)$, es decir, la manera en la que se modifica el tamaño del problema en la recursión: **recurrencias por sustracción** y **recurrencias por división**. Por otro lado, en esta asignatura no abordaremos los diferentes métodos existentes de resolución de recurrencias y nos limitaremos a obtener el coste de los dos tipos de recurrencias anteriores aplicando resultados matemáticos conocidos.

Recurrencias por sustracción

Estas recurrencias se caracterizan porque los tamaños de los subproblemas consisten en restas de ciertas cantidades constantes, es decir: $b_i(n) = n - b_i$, con $b_i \in \mathbb{N}^+$. El mayor tamaño de subproblema es $n - b$ donde $b = \min\{b_i \mid i = 1, 2, \dots, k\}$ siendo k el número de tamaños de subproblemas diferentes.

El Teorema 1 nos permite obtener una cota superior de una recurrencia por sustracción:

Teorema 1 (Teorema Maestro para Recurrencias por Sustracción). *Dada una recurrencia por sustracción:*

$$T_{\mathcal{A}}(n) = \begin{cases} T_{cb}(n) & \text{si } 0 \leq n < b \\ a \cdot T_{\mathcal{A}}(n - b) + T_{nr}(n) & \text{si } n \geq b \end{cases}$$

Se cumple que dicha recurrencia se acota superiormente de la siguiente manera:

$$T_{\mathcal{A}}(n) \in \begin{cases} \mathcal{O}(n \cdot T_{nr}(n) + T_{cb}(n)) & \text{si } a = 1 \\ \mathcal{O}(a^{(n \text{ div } b)} \cdot (T_{nr}(n) + T_{cb}(n))) & \text{si } a > 1 \end{cases}$$

La demostración se realiza desplegando la recursión y distinguiendo casos según el valor del número de llamadas recursivas a . No obstante, se omite en estos apuntes.

Un caso habitual se tiene cuando el coste del caso base (o casos base) es (son) constante(s), i.e. $T_{cb}(n) \in \Theta(1)$ y el coste de las operaciones no recursivas son constantes o polinomiales, es decir, $T_{nr}(n) \in \Theta(n^k)$ con $k \in \mathbb{N}_0$. Nótese que si $k = 0$ se tiene coste constante pues $\Theta(n^0) = \Theta(1)$. Entonces:

$$T_{\mathcal{A}}(n) = \begin{cases} \Theta(1) & \text{si } 0 \leq n < b \\ a \cdot T_{\mathcal{A}}(n - b) + \Theta(n^k) & \text{si } n \geq b \end{cases}$$

En este caso se tiene lo siguiente:

$$T_{\mathcal{A}}(n) \in \begin{cases} \Theta(n^{k+1}) & \text{si } a = 1 \\ \Theta(a^{(n \text{ div } b)} \cdot n^k) & \text{si } a > 1 \end{cases}$$

Volviendo al ejemplo del n -ésimo número de Fibonacci:

$$T_{fib}(n) = \begin{cases} \Theta(1) & \text{si } n \leq 1 \\ T_{fib}(n - 1) + T_{fib}(n - 2) + \Theta(1) & \text{si } n > 1 \end{cases}$$

Se reescribe con la forma simplificada de esta manera:

$$T_{fib}(n) = \begin{cases} \Theta(1) & \text{si } n \leq 1 \\ 2 \cdot T_{fib}(n-1) + \Theta(1) & \text{si } n > 1 \end{cases}$$

El número de llamadas recursivas es $a = 2$. El mayor tamaño de subproblemas es $n-1 > n-2$, luego $n-b = n-1$, por tanto $b = 1$. Como se indicó anteriormente, $T_{cb}, T_{nr} \in \Theta(1)$. Aplicando el teorema anterior, tenemos que $a = 2 > 1$, luego $T_{fib}(n) \in \mathcal{O}(a^{(n \text{ div } b)} \cdot (T_{nr}(n) + T_{cb}(n)))$. Sustituyendo: $T_{fib}(n) \in \mathcal{O}(2^{(n \text{ div } 1)} \cdot (\Theta(1) + \Theta(1))) = \mathcal{O}(2^n)$, lo cual, también encaja con el caso habitual. Al haber asumido dos subproblemas de tamaño $n-1$ en lugar de uno de tamaño $n-1$ y otro de tamaño $n-2$, esta no es la cota más precisa: se trata de $\mathcal{O}(\varphi^n)$ siendo $\varphi = \frac{1+\sqrt{5}}{2} = 1,618 \dots$ el *número áureo* (o *razón áurea*), tasa de crecimiento de la sucesión de Fibonacci a medida que crece el valor de n .

Recurrencias por división¹²

Estas recurrencias se caracterizan porque los tamaños de los subproblemas consisten en divisiones de ciertas cantidades constantes, es decir: $b_i(n) = \frac{n}{b_i}$, con $b_i \in \mathbb{N}^+$ tales que $b_i > 1$. El mayor tamaño de subproblema es $n \text{ div } b$ donde $b = \min\{b_i \mid i = 1, 2, \dots, k\}$ siendo k el número de tamaños de subproblemas diferentes.

El Teorema 2 nos permite obtener una cota superior de una recurrencia por división:

Teorema 2 (Teorema Maestro para Recurrencias por División). *Dada una recurrencia por división:*

$$T_A(n) = \begin{cases} T_{cb}(n) & \text{si } 1 \leq n < b \\ a \cdot T_A(n \text{ div } b) + T_{nr}(n) & \text{si } n \geq b \end{cases}$$

Se cumple que dicha recurrencia se acota superiormente de la siguiente manera:

$$T_A(n) \in \begin{cases} \mathcal{O}(\log_b n \cdot T_{nr}(n) + T_{cb}(n)) & \text{si } a = 1 \\ \mathcal{O}(n^{\log_b a} \cdot (T_{nr}(n) + T_{cb}(n))) & \text{si } a > 1 \end{cases}$$

Al igual que en el caso de las recurrencias por sustracción, en estos apuntes omitimos la demostración del anterior teorema.

El anterior teorema proporciona cotas superiores válidas pero no las más ajustadas. Sin embargo, en esta asignatura afinaremos más el análisis para el caso habitual análogo visto para recurrencias por sustracción: $T_{cb}(n) \in \Theta(1)$ y $T_{nr}(n) \in \Theta(n^k)$ con $k \in \mathbb{N}_0$. La recurrencia tiene la forma:

$$T_A(n) = \begin{cases} \Theta(1) & \text{si } 0 \leq n < b \\ a \cdot T_A(n \text{ div } b) + \Theta(n^k) & \text{si } n \geq b \end{cases}$$

En este caso se tiene lo siguiente:

¹²Suelen ser recurrencias asociadas a algoritmos que siguen la estrategia *divide y vencerás* (*divide and conquer* en inglés), de manera que también suelen denominarse *recurrencias divide y vencerás*.

$$T_{\mathcal{A}}(n) \in \begin{cases} \Theta(n^k) & \text{si } a < b^k \\ \Theta(n^k \cdot \log n) & \text{si } a = b^k \\ \Theta(n^{\log_b a}) & \text{si } a > b^k \end{cases}$$

Pongamos un ejemplo:

$$T_{\mathcal{A}}(n) = \begin{cases} \Theta(1) & \text{si } n = 1 \\ 7 \cdot T_{\mathcal{A}}(n \operatorname{div} 2) + \Theta(n^2) & \text{si } n > 1 \end{cases}$$

Identificamos los parámetros:

- Número de llamadas recursivas $a = 7$.
- Factor de reducción de subproblemas: $b = 2$.
- Coste caso base: $T_{cb}(n) \in \Theta(1)$.
- Coste ops. no recursivas: $T_{nr}(n) \in \Theta(n^2)$.

Este ejemplo encaja en el caso habitual con $k = 2$, dado que $T_{cb}(n) \in \Theta(1)$ y $T_{nr}(n) \in \Theta(n^2)$, por tanto hay que comparar los valores a y b^k : $a = 7 > 4 = 2^2 = b^k$. Por tanto, se tiene que $T_{\mathcal{A}}(n) \in \Theta(n^{\log_b a}) = \Theta(n^{\log_2 7}) = \Theta(n^{2,807\dots})$. Esta es la recurrencia asociada al *algoritmo de Strassen*¹³ de multiplicación de matrices mediante la estrategia *divide y vencerás*. Este algoritmo es más eficiente que el algoritmo de multiplicación de matrices estándar por fuerza bruta cuya complejidad es $\Theta(n^3)$, equivalente a tener $a = 8$ llamadas recursivas, dado que $\Theta(n^{\log_2 8}) = \Theta(n^3)$.

¹³Llamado así por su creador, el matemático alemán Volker Strassen.

Bibliografia

- [1] C. A. R. Hoare, “An axiomatic basis for computer programming,” *Commun. ACM*, vol. 12, no. 10, p. 576–580, Oct. 1969. [Online]. Available: <https://doi.org/10.1145/363235.363259>
- [2] C. B. Boyer and U. C. Merzbach, *A history of mathematics*. John Wiley & Sons, 2011.